

Network Modeling: Assignment 2

Group number: 21

Group members: Eduard Mihai, Hanzhang Wang, Silvan Affentranger, Linus Michel

Libraries

```
# Install packages from CRAN
install.packages(
  c("network", "ergm", "manynet", "patchwork", "latticeExtra", "texreg", "broom")
)
# Install autograph from GitHub develop branch
# install.packages("remotes") # if not already installed
remotes::install_github("stocnet/autograph", ref = "develop")
tinytex::install_tinytex()
```

```
# Loading libraries - stocnet suite
library(manynet)
library(autograph)
# Loading libraries - sna suite
library(network)
```

```
##
## 'network' 1.20.0 (2026-02-06), part of the Statnet Project
## * 'news(package="network")' for changes since last version
## * 'citation("network")' for citation information
## * 'https://statnet.org' for help, support, and other information
```

```
library(ergm)
```

```
##
## 'ergm' 4.12.0 (2026-02-17), part of the Statnet Project
## * 'news(package="ergm")' for changes since last version
## * 'citation("ergm")' for citation information
## * 'https://statnet.org' for help, support, and other information
```

```
## 'ergm' 4 is a major update that introduces some backwards-incompatible
## changes. Please type 'news(package="ergm")' for a list of major
## changes.
```

```
# Loading libraries - plotting and presentation
library(ggraph)
```

```
## Lade nötiges Paket: ggplot2
```

```
library(patchwork)
# library(latticeExtra) # improve the aesthetics of the MCMC diagnostics plots
library(texreg)
```

```
## Version: 1.39.5
## Date: 2025-12-21
## Author: Philip Leifeld (University of Manchester)
##
## Consider submitting praise using the praise or praise_interactive functions.
## Please cite the JSS article in your publications -- see citation("texreg").
```

```
library(broom)
library(sna)
```

```
## Lade nötiges Paket: statnet.common
```

```
##
## Attache Paket: 'statnet.common'
```

```
## Die folgenden Objekte sind maskiert von 'package:base':
##
## attr, order, replace
```

```
## sna: Tools for Social Network Analysis
## Version 2.8 created on 2024-09-07.
## copyright (c) 2005, Carter T. Butts, University of California-Irvine
## For citation information, type citation("sna").
## Type help(package="sna") to get started.
```

```
library(xtable)
library(ggplot2)
library(tidyr)
```

```
##
## Attache Paket: 'tidyr'
```

```
## Das folgende Objekt ist maskiert 'package:texreg':
##
## extract
```

```
library(parallel)
```

Task 1: Geometrically Weighted Terms in ERGMs

1.1

1.1.i

When a directed edge is added to a sender node with current out-degree k , exactly two terms in the GWD statistic are affected: the count of nodes with out-degree k decreases by one, and the count of nodes

with out-degree $k+1$ increases by one. Thus, if we take the difference $s^{\text{GWD}}(x', \alpha) - s^{\text{GWD}}(x, \alpha)$, all terms in the sum except k and $k+1$ cancel out. Then we write out what is left giving us $e^\alpha \left[\left(1 - (1 - e^{-\alpha})^{k+1}\right) - \left(1 - (1 - e^{-\alpha})^k\right) \right]$ and cancel the two ones: $e^\alpha \left[(1 - e^{-\alpha})^k - (1 - e^{-\alpha})^{k+1} \right]$. Next we factor out $(1 - e^{-\alpha})^k$ leaving us with $e^\alpha (1 - e^{-\alpha})^k [1 - (1 - e^{-\alpha})]$. Finally we realize $1 - (1 - e^{-\alpha}) = e^{-\alpha}$ and thus we get $\delta^{\text{GWD}}(k, \alpha) = (1 - e^{-\alpha})^k$.

$$\begin{aligned}
\delta^{\text{GWD}}(k, \alpha) &= s^{\text{GWD}}(x', \alpha) - s^{\text{GWD}}(x, \alpha) \\
&= e^\alpha \left[\left(1 - (1 - e^{-\alpha})^{k+1}\right) - \left(1 - (1 - e^{-\alpha})^k\right) \right] \\
&= e^\alpha \left[(1 - e^{-\alpha})^k - (1 - e^{-\alpha})^{k+1} \right] \\
&= e^\alpha (1 - e^{-\alpha})^k [1 - (1 - e^{-\alpha})] \\
&= e^\alpha (1 - e^{-\alpha})^k \cdot e^{-\alpha} \\
&= \boxed{(1 - e^{-\alpha})^k}
\end{aligned}$$

1.1.ii

```

f1 <- as.matrix(read.csv("Glasgow/f1.csv", header = FALSE))
f2 <- as.matrix(read.csv("Glasgow/f2.csv", header = FALSE))
f3 <- as.matrix(read.csv("Glasgow/f3.csv", header = FALSE))

demographic <- read.csv("Glasgow/demographic.csv")
logdistance <- as.matrix(read.csv("Glasgow/logdistance.csv", header = FALSE))
alcohol <- read.csv("Glasgow/alcohol.csv")

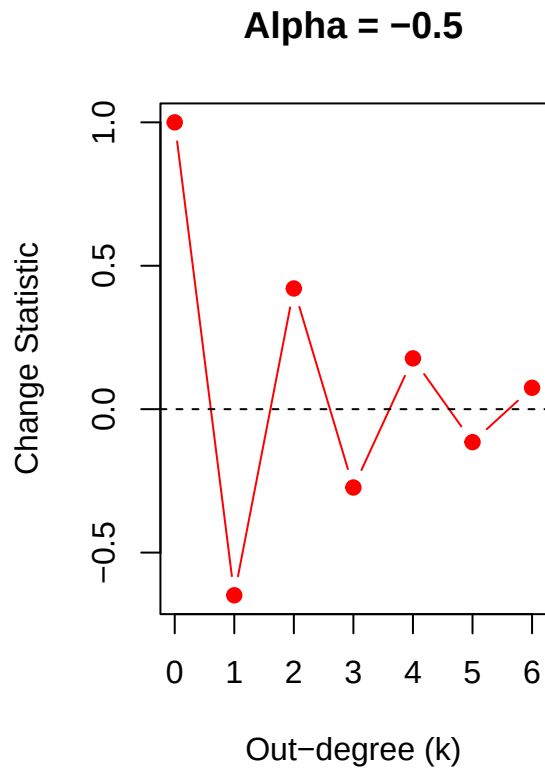
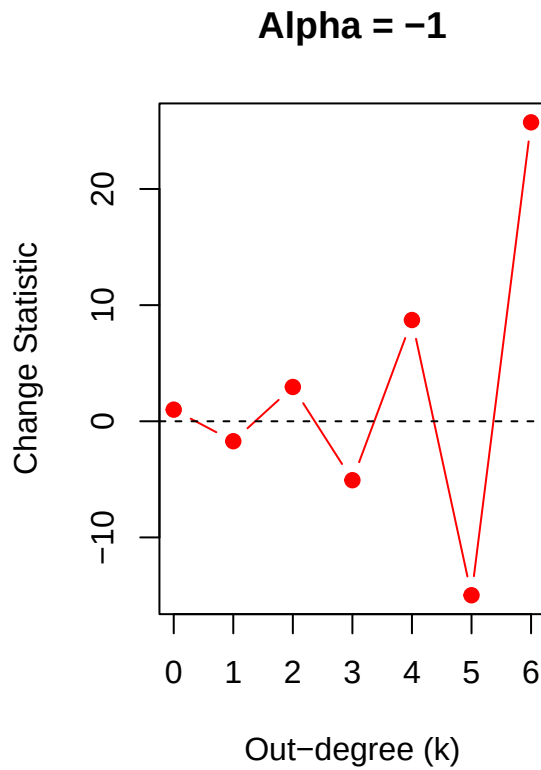
# out degree range from f1
max_k <- max(rowSums(f1))
k_vals <- 0:max_k

# change statistic function
change_stat <- function(k, a) {
  if (a == 0) {
    return(ifelse(k==0, 1, 0))
  }
  return((1 - exp(-a))^k)
}

# alpha values to test
decay_val_neg <- c(-1, -0.5)
decay_val_pos <- c(0, 0.1, 0.2, 0.3, 0.5, log(2), 1, 2, 3)

# generate plots for negative values of alpha
par(mfrow = c(1,2))
for (a in decay_val_neg) {
  y_vals <- sapply(k_vals, change_stat, a = a)
  plot(k_vals, y_vals, type = "b", pch = 19, col = "red",
       main = paste("Alpha =", a), xlab = "Out-degree (k)", ylab = "Change Statistic")
  abline(h = 0, lty = 2)
}

```

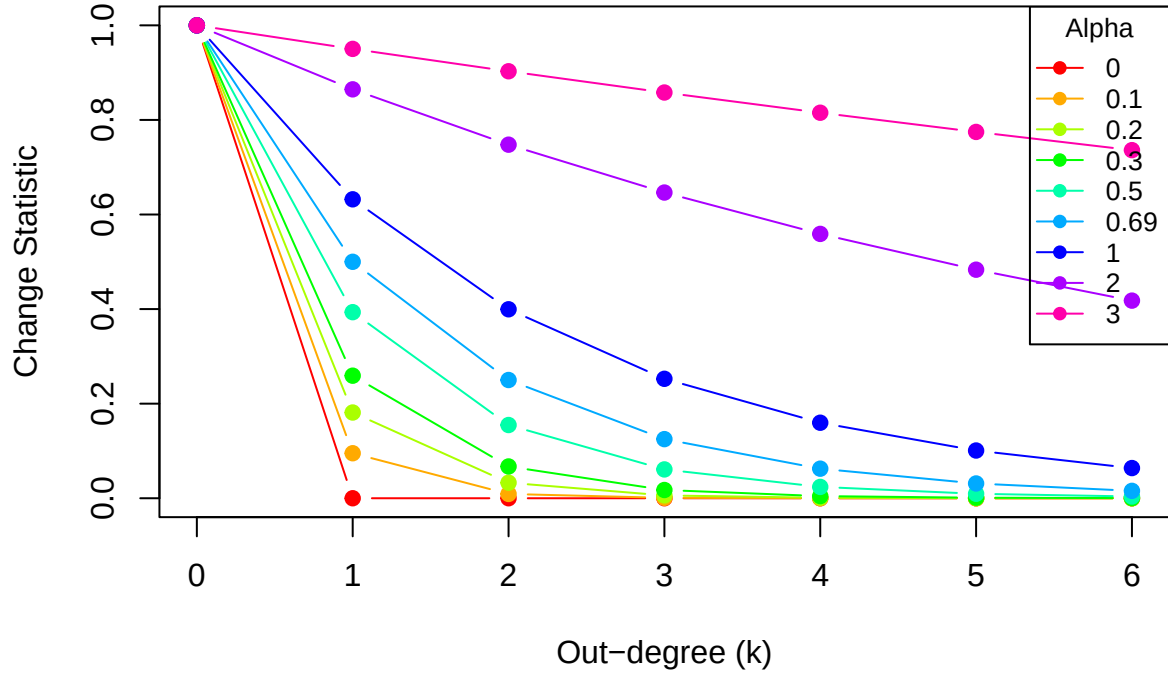


```
# generate plots for positive values of alpha
par(mfrow = c(1, 1))
colors <- rainbow(length(decay_val_pos))
plot(NULL, xlim = c(0, max_k), ylim = c(0, 1), xlab = "Out-degree (k)",
      ylab = "Change Statistic", main = "GWD Change Statistic (Non-negative Alpha)")

for (i in 1:length(decay_val_pos)) {
  y_vals <- sapply(k_vals, change_stat, a = decay_val_pos[i])
  lines(k_vals, y_vals, type = "b", pch = 19, col = colors[i])
}
```

```
legend("topright", legend = round(decay_val_pos, 2), col = colors, lty = 1, pch = 19, title = "Alpha", c
```

GWD Change Statistic (Non-negative Alpha)



1.1.iii

The figures from the previous task show us that the decay parameter is the primary influence on the diminishing marginal returns principle when using `gwdegree`. If we look at the change statistics, we can get the following takeaways, regarding how additional edges influence already popular nodes. The first takeaway is that for all non-negative decay values, the marginal contribution of an additional edge is strictly non-increasing. As can be seen in our plot, the weight added to the network statistic by a new edge is always maximized when the node where the edge originates has an out-degree of zero. As k increases, this contribution decreases, meaning that with each additional edge a node has, the weights of that nodes connections decrease. With the all-positive decay values, the value of α gives us the speed of this decrease (as can be seen in the plot). With small values of α , the change statistic drops to zero shortly after the first couple of edges, while the marginal contribution of nodes with higher out-degrees stays high with larger values of α . This allows us to choose how influential we want high-degree hub-nodes to be on the overall network statistic. The second takeaway is about the negative values for the decay parameter α . For negative values it has a very unstable behavior, as we can see in the way its graphs jump around. This is, because the base of our change statistic becomes negative and thus oscillates between positive and negative values, as the out-degree increases. This means that if a second edge is added to a node, this second tie would decrease the change statistic, while adding a third such tie would increase it again. Applied to real world scenarios, this makes little sense, which tells us that a positive α as decay parameter value is needed to have meaningful results.

1.1.iv

To capture an activity effect where high out-degree nodes drive the statistic, we need to choose the decay parameter α to be relatively high. How high exactly depends on how high the out-degree nodes we want

to drive the statistic are. As we can see in the plot of positive values for alpha, the choice of alpha influences the rate of diminishing marginal returns for each additional outgoing tie. When alpha is small, the change statistic drops to zero almost immediately after the first tie. The model essentially only distinguishes between isolates and non-isolates, which means that a small alpha would thus not effectively capture the behavior of high-activity nodes because their additional ties would contribute only a very small amount to the network statistic. If we choose a larger alpha (like 2 or 3 for example), the curve of the change statistic flattens, which causes the marginal contribution of an extra tie to remain substantial even as a node's out-degree increases. This means that for a student in our dataset for example, if he already named four friends, naming the fifth and even a sixth friend still adds significant weight to the network statistic. This prevents the model from saturating early and maintains that additional ties from already active keep having marginal impact, which gives us our desired outcome, where high out-degree nodes drive the statistic.

1.2

1.2.i

```
f1_matrix <- as.matrix(read.csv("Glasgow/f1.csv", header = FALSE))
glasgow_net <- as.network(f1_matrix, directed = TRUE)

max_k <- max(rowSums(f1_matrix))

m1 <- ergm(glasgow_net ~ edges)
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
```

```
## Obtaining the responsible dyads.
```

```
## Evaluating the predictor and response matrix.
```

```
## Maximizing the pseudolikelihood.
```

```
## Finished MPLE.
```

```
## Evaluating log-likelihood at the estimate.
```

1.2.ii

```
m2_neg <- ergm(glasgow_net ~ edges + offset(gwodegree(0.3, fixed = TRUE)),
               offset.coef = c(-2))
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
```

```
## Obtaining the responsible dyads.
```

```
## Evaluating the predictor and response matrix.
```

```
## Maximizing the pseudolikelihood.
```

```

## Finished MPLE.

## Starting Monte Carlo maximum likelihood estimation (MCMLE):

## Iteration 1 of at most 60:

## Warning: 'glpk' selected as the solver, but package 'Rglpk' is not available;
## falling back to 'lpSolveAPI'. This should be fine unless the sample size and/or
## the number of parameters is very big.

## 1

## Optimizing with step length 0.9271.

## The log-likelihood improved by 3.6667.

## Estimating equations are not within tolerance region.

## Iteration 2 of at most 60:

## 1 Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0080.
## Convergence test p-value: 0.0075. Converged with 99% confidence.
## Finished MCMLE.
## Evaluating log-likelihood at the estimate. Fitting the dyad-independent submodel...
## Bridging between the dyad-independent submodel and the full model...
## Setting up bridge sampling...
## Using 16 bridges: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 .
## Bridging finished.
##
## This model was fit using MCMC. To examine model diagnostics and check
## for degeneracy, use the mcmc.diagnostics() function.

m2_pos <- ergm(glasgow_net ~ edges + offset(gwodegree(0.3, fixed = TRUE)),
              offset.coef = c(2))

## Starting maximum pseudolikelihood estimation (MPLE):
## Obtaining the responsible dyads.
## Evaluating the predictor and response matrix.
## Maximizing the pseudolikelihood.
## Finished MPLE.
## Starting Monte Carlo maximum likelihood estimation (MCMLE):
## Iteration 1 of at most 60:
## 1 Optimizing with step length 0.6214.
## The log-likelihood improved by 2.5265.
## Estimating equations are not within tolerance region.
## Iteration 2 of at most 60:
## 1 Optimizing with step length 1.0000.
## The log-likelihood improved by 0.2304.
## Estimating equations are not within tolerance region.
## Iteration 3 of at most 60:

```

```

## 1 Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0100.
## Convergence test p-value: 0.0082. Converged with 99% confidence.
## Finished MCMLE.
## Evaluating log-likelihood at the estimate. Fitting the dyad-independent submodel...
## Bridging between the dyad-independent submodel and the full model...
## Setting up bridge sampling...
## Using 16 bridges: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 .
## Bridging finished.
##
## This model was fit using MCMC. To examine model diagnostics and check
## for degeneracy, use the mcmc.diagnostics() function.

```

1.2.iii

```

sim_out_dist <- function(formula, coefficients, n_sim = 100, max_val) {
  sims <- ergm::simulate_formula(formula, coef = coefficients, nsim = n_sim)

  dist_list <- lapply(sims, function(x) {
    deg_counts <- table(factor(rowSums(as.matrix(x)), levels = 0:max_val))
    return(as.data.frame(t(as.matrix(deg_counts))))
  })

  return(do.call(rbind, dist_list))
}

set.seed(123)

# M1
sim_m1_df <- sim_out_dist(glasgow_net ~ edges,
                          coef(m1), 100, max_k)

# M2 negative gwd
sim_m2_neg_df <- sim_out_dist(glasgow_net ~ edges + offset(gwodegree(0.3, fixed = TRUE)),
                              coef(m2_neg), 100, max_k)

# M2 positive gwd
sim_m2_pos_df <- sim_out_dist(glasgow_net ~ edges + offset(gwodegree(0.3, fixed = TRUE)),
                              coef(m2_pos), 100, max_k)

# formatting for plot
process_df <- function(df, label) {
  df %>%
    pivot_longer(cols = everything(), names_to = "degree", values_to = "node_count") %>%
    mutate(model = label, degree = as.numeric(degree))
}

# combine data
plot_data <- rbind(
  process_df(sim_m1_df, "M1 (Edges Only)"),
  process_df(sim_m2_neg_df, "M2 (GWOD Negative)"),
  process_df(sim_m2_pos_df, "M2 (GWOD Positive)")
)

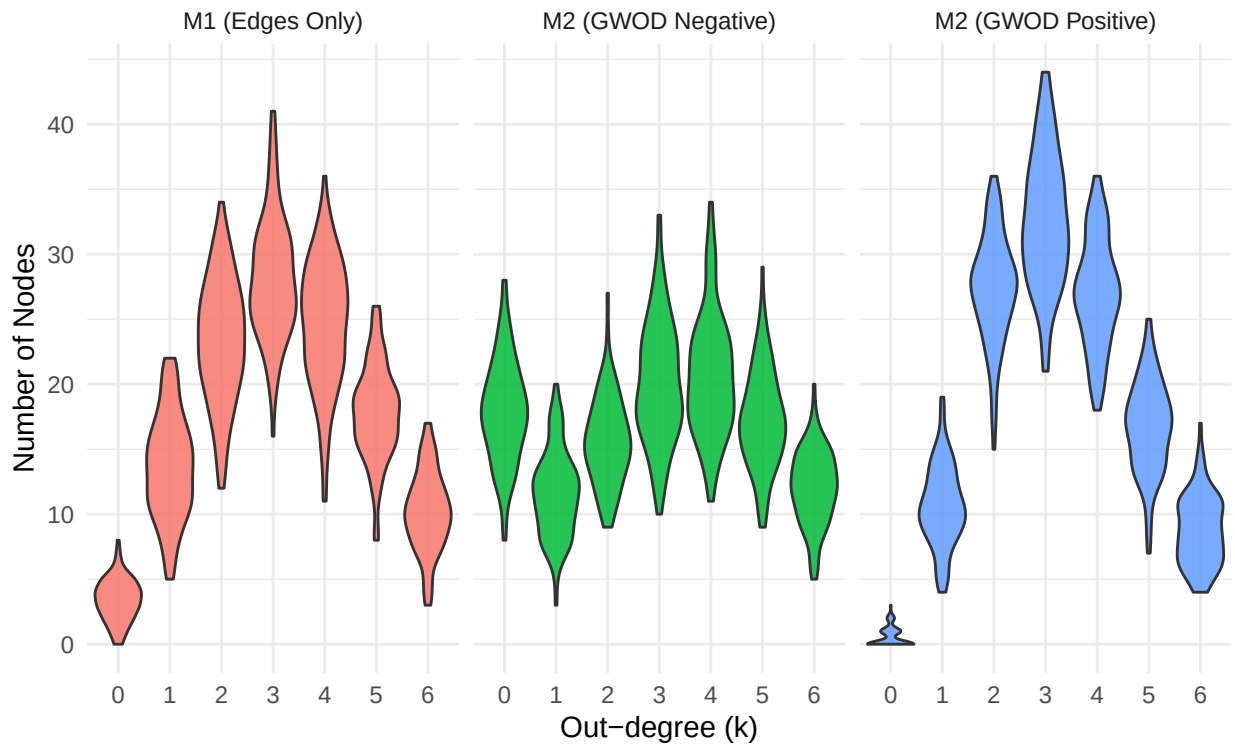
```



```
# generate plot
ggplot(plot_data, aes(x = factor(degree), y = node_count, fill = model)) +
  geom_violin(trim = TRUE, scale = "width", alpha = 0.85) +
  facet_wrap(~model) +
  labs(
    title = "Out-Degree Distribution Comparison",
    subtitle = "Decay = 0.3, 100 Simulations",
    x = "Out-degree (k)",
    y = "Number of Nodes"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

Out-Degree Distribution Comparison

Decay = 0.3, 100 Simulations



1.2.iv

As we can see in the plot, the gwodegree alters the structural tendencies of a network compared to a random baseline network. By comparing the three different configurations, we can observe how different gwodegrees influence the degree variance among nodes. The baseline is our M1 configuration, which represents a Bernoulli random network where the probability of a tie is uniformly spread across all node pairs. As we can see, the out-degree distribution follows a typical bell-curve, that peaks at an out-degree of three. This indicates that most nodes have a similar number of outgoing ties and the extremes, nodes having zero or six ties, are infrequent. In M2 with a negative gwodegree, we can observe that with a negative gwodegree the distribution is flatter and more dispersed than in the random baseline. This is, because the model penalizes higher out-degrees if it has a negative gwodegree. Another obvious difference to the random baseline is, that

the extremes of having six and especially zero ties are much more likely. If we take a look at M2 with a positive gwodegree, we can see from the plot, that it increases the number of nodes with multiple outgoing ties. In this configuration, the distribution becomes much narrower and taller and is concentrated around out-degrees of two, three and four. What can also easily be noticed is that the number of isolated nodes with an out-degree of zero drops to almost zero.

Task 2: Estimation and interpretation of an ERGM

2.1

```
# Reading the adjacency matrices using manynet::read_matrix()
friendship <- read_matrix("Glasgow/f1.csv", header = FALSE)
demographic <- read.csv("Glasgow/demographic.csv", header = TRUE)

net <- network(as.matrix(friendship), directed = TRUE)
net %v% "gender" <- as.character(demographic[, "gender"])
set.seed(161)
# this from the last assignment: permutations <- 5000
# estimate ERGM with parameters of tie existence and if that tie is same gender
model <- ergm(net ~ edges + nodematch("gender"), control = control.ergm(seed = 161))
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
```

```
## Obtaining the responsible dyads.
```

```
## Evaluating the predictor and response matrix.
```

```
## Maximizing the pseudolikelihood.
```

```
## Finished MPLE.
```

```
## Evaluating log-likelihood at the estimate.
```

```
#saveRDS(model, "model.rds")
summary(model)
```

```
## Call:
```

```
## ergm(formula = net ~ edges + nodematch("gender"), control = control.ergm(seed = 161))
```

```
##
```

```
## Maximum Likelihood Results:
```

```
##
```

```
##           Estimate Std. Error MCMC % z value Pr(>|z|)
## edges          -5.0133    0.1365      0 -36.72  <1e-04 ***
## nodematch.gender  2.0098    0.1460      0  13.77  <1e-04 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## Null Deviance: 22890 on 16512 degrees of freedom
```

```
## Residual Deviance: 3824 on 16510 degrees of freedom
```

```
##
```

```
## AIC: 3828 BIC: 3843 (Smaller is better. MC Std. Err. = 0)
```

```

#get conditional probs -----
#probability that a tie exists between two nodes
theta_edges <- coef(model)["edges"]
#additional chance a tie exists if the nodes are same gender
theta_match <- coef(model)["nodematch.gender"]

# Log-odds of a tie between two same-gender students
logodds_same <- theta_edges + theta_match

# Converted to probability (exp / (1 + exp))
prob_same <- plogis(logodds_same)

# same for different gender tie (nodematch = 0)
logodds_diff <- theta_edges
prob_diff <- plogis(logodds_diff)

cat(sprintf(" P(tie | same gender)      :  %.4f  (%.1f %%)\n", prob_same, prob_same * 100))

##   P(tie | same gender)      :  0.0473  (4.7 %)

cat(sprintf(" P(tie | diff gender)      :  %.4f  (%.1f %%)\n", prob_diff, prob_diff * 100))

##   P(tie | diff gender)      :  0.0066  (0.7 %)

cat(sprintf(" Ratio Same-Diff gender    :  %.4f  \n", prob_same/prob_diff))

##   Ratio Same-Diff gender    :  7.1563

```

Both results hold strong evidence that edges are both sparse ($\theta_{edges} = -5.0133$) and homophilic ($\theta_{match} = 2.0098$), as $p < 0.05$ in both cases. Although there is only a probability of $P_{same} = 4.7\%$ for a tie to exist between two students of the same gender, it is still 7.1563 times higher than the probability of a tie existing between two students of opposite gender $P_{diff} = 0.7\%$. The huge difference in Null Deviance to Residual Deviance ($22890 \rightarrow 3824$) also tells that the tie structure is very dependent on the chosen parameters.

2.2

To build the big ERGM model similar to 2.1:

```

alcohol <- read.csv("Glasgow/alcohol.csv", header = TRUE)

# Attach just the first column
net %v% "alcohol" <- alcohol[, 1]

#this takes a lil while:
big_model <- ergm(net ~ edges + nodematch("gender") #same baseline as 2.1
  + mutual #(i) Reciprocity
  + gwespc(0.1, fixed = TRUE) #(ii) Transitivity
  + gwodegree(0.1, fixed = TRUE) # controls for out-degree distribution

```

```

+ gwidegree(0.1, fixed = TRUE)      # controls for in-degree distribution

+ nodeofactor("gender", base = 2) # (iii) Boys have higher node outdegree
+ nodeifactor("gender", base = 2) # (iv) Boys have higher node indegree
+ nodeocov("alcohol") # (v) high alcohol -> high node outdegree
+ nodeicov("alcohol") # (vi) high alcohol -> high node indegree
+ absdiff("alcohol"), # (vii) alcohol homophily
control = control.ergm(seed = 161, MCMLE.maxit = 40, MCMC.samplesize = 2048,
  MCMC.burnin = 10000, MCMC.interval = 500)
)
#summary(big_model)
#mcmc.diagnostics(big_model)
#saveRDS(big_model, "big_model.rds")

bigger_model <- ergm(net ~ edges + nodematch("gender") #same baseline as 2.1
+ mutual # (i) Reciprocity
+ gwesp(0.1, fixed = TRUE) # (ii) Transitivity

+ gwodegree(0.1, fixed = TRUE)      # controls for out-degree distribution
+ gwidegree(0.1, fixed = TRUE)      # controls for in-degree distribution

+ nodeofactor("gender", base = 2) # (iii) Boys have higher node outdegree
+ nodeifactor("gender", base = 2) # (iv) Boys have higher node indegree
+ nodeocov("alcohol") # (v) high alcohol -> high node outdegree
+ nodeicov("alcohol") # (vi) high alcohol -> high node indegree
+ absdiff("alcohol"), # (vii) alcohol homophily
control = control.ergm(seed = 161, MCMC.samplesize = 50000, init = coef(big_model),
  MCMC.burnin = 100000, MCMC.interval = 2000, MCMLE.maxit = 60)
)
saveRDS(bigger_model, "bigger_model.rds")

bigger_model <- readRDS("bigger_model.rds")
summary(bigger_model)

```

```

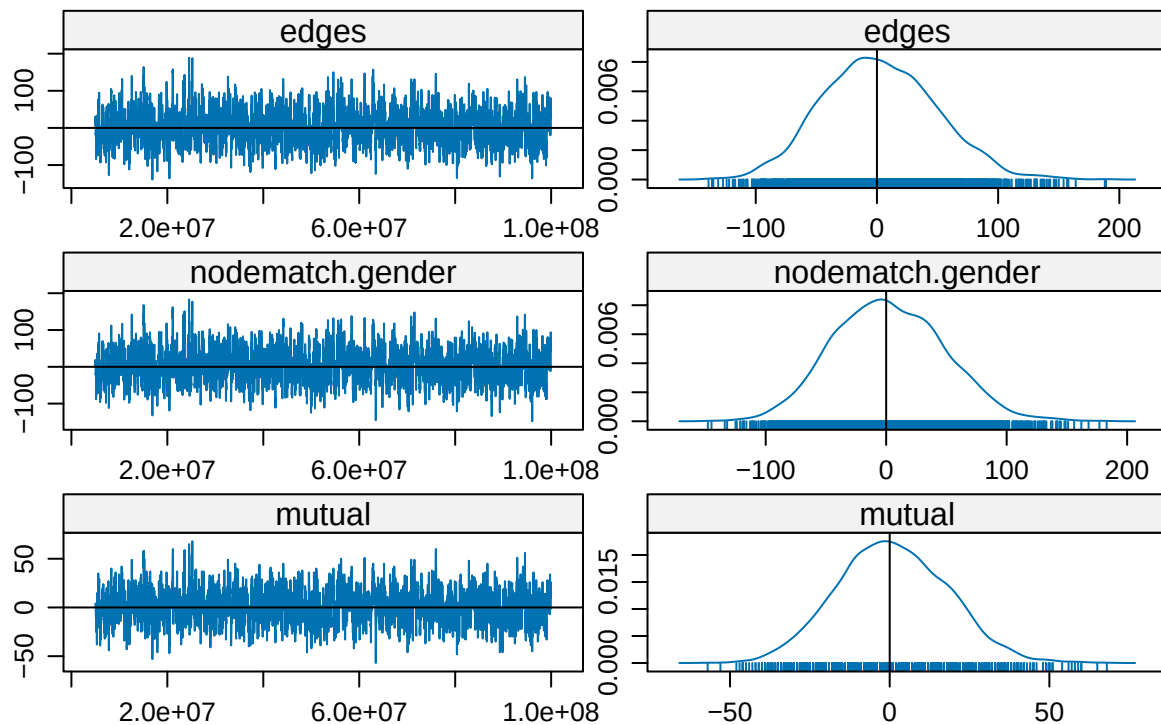
## Call:
## ergm(formula = net ~ edges + nodematch("gender") + mutual + gwesp(0.1,
##   fixed = TRUE) + gwodegree(0.1, fixed = TRUE) + gwidegree(0.1,
##   fixed = TRUE) + nodeofactor("gender", base = 2) + nodeifactor("gender",
##   base = 2) + nodeocov("alcohol") + nodeicov("alcohol") + absdiff("alcohol"),
##   control = control.ergm(seed = 161, MCMC.samplesize = 50000,
##     init = coef(big_model), MCMC.burnin = 1e+05, MCMC.interval = 2000,
##     MCMLE.maxit = 60))
##
## Monte Carlo Maximum Likelihood Results:
##
##               Estimate Std. Error MCMC % z value Pr(>|z|)
## edges          -5.95141    0.17513    0 -33.983 < 1e-04 ***
## nodematch.gender    0.85499    0.10496    0  8.146 < 1e-04 ***
## mutual           3.02297    0.19107    0 15.821 < 1e-04 ***
## gwesp.OTP.fixed.0.1  1.67667    0.10814    0 15.505 < 1e-04 ***
## gwodeg.fixed.0.1    1.84220    0.48991    0  3.760 0.000170 ***
## gwideg.fixed.0.1    1.60488    0.44954    0  3.570 0.000357 ***
## nodeofactor.gender.1 -0.10194    0.15031    0 -0.678 0.497628

```

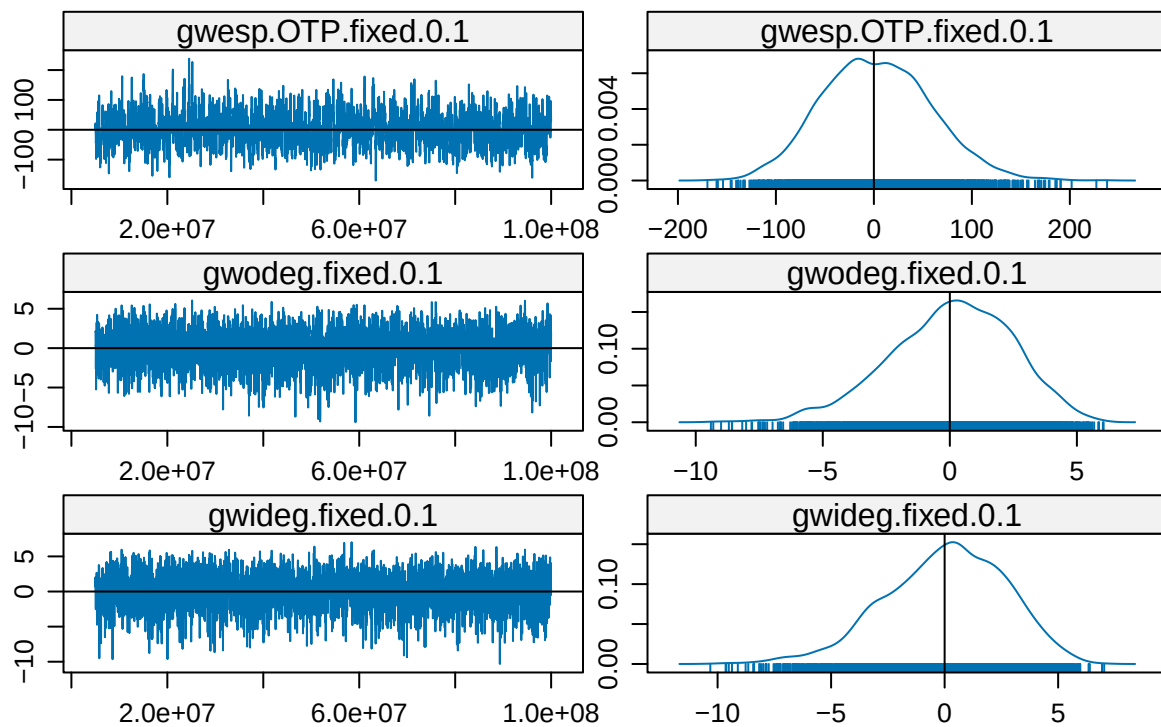
```
## nodeifactor.gender.1  0.03781    0.14949    0  0.253 0.800318
## nodecov.alcohol      -0.11891    0.05540    0 -2.147 0.031823 *
## nodeicov.alcohol     0.15648    0.05472    0  2.860 0.004242 **
## absdiff.alcohol      -0.11072    0.03710    0 -2.985 0.002840 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 22890  on 16512  degrees of freedom
## Residual Deviance:  2845  on 16501  degrees of freedom
##
## AIC: 2867  BIC: 2952  (Smaller is better. MC Std. Err. = 0.6422)
```

```
mcmc.diagnostics(bigger_model)
```

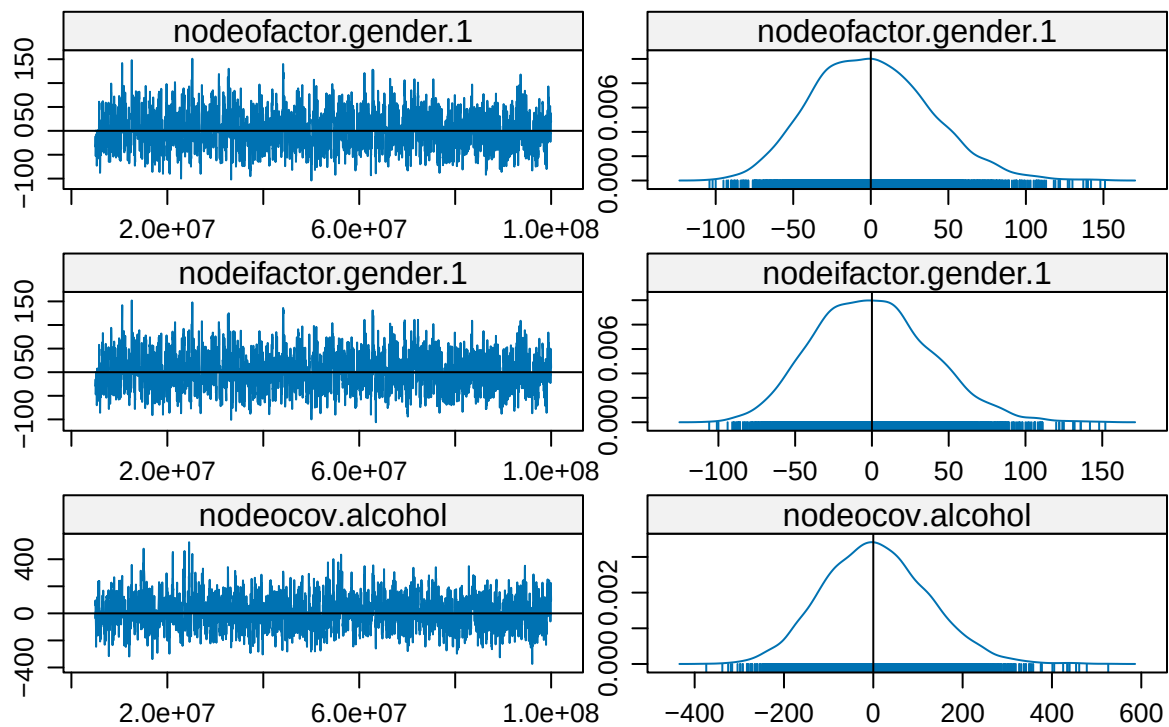
Sample statistics



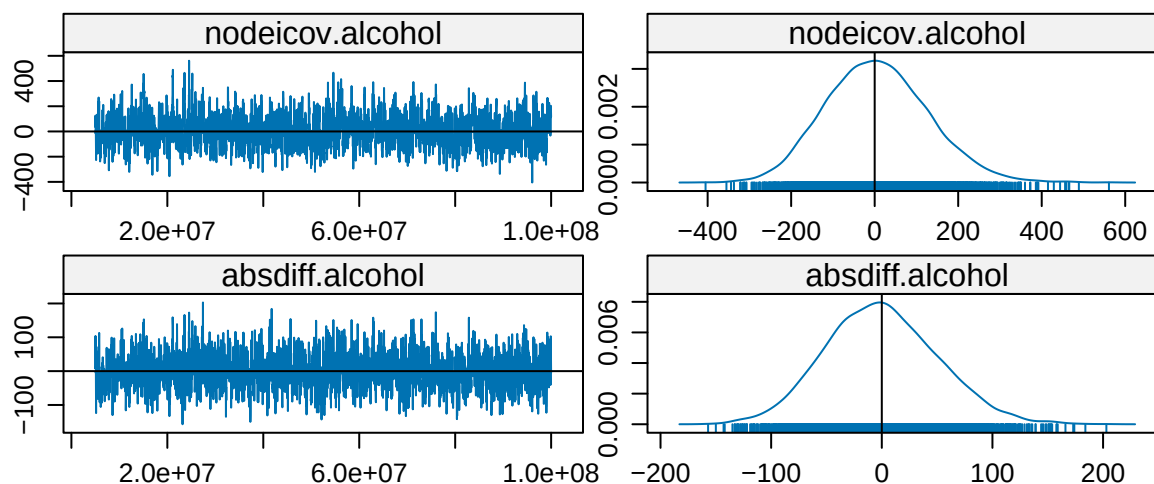
Sample statistics



Sample statistics



Sample statistics



Sample statistics summary:

##

Iterations = 5002000:99994000

Thinning interval = 24000

Number of chains = 1

Sample size per chain = 3959

##

1. Empirical mean and standard deviation for each variable,
plus standard error of the mean:

##

	Mean	SD	Naive SE	Time-series SE
## edges	2.27936	46.589	0.74045	1.63933
## nodematch.gender	2.28416	45.868	0.72898	1.56563
## mutual	0.91917	17.175	0.27297	0.60137
## gwesp.OTP.fixed.0.1	2.96813	55.530	0.88255	1.87951
## gwodeg.fixed.0.1	0.04214	2.406	0.03824	0.05035
## gwideg.fixed.0.1	0.02785	2.643	0.04201	0.05697
## nodeofactor.gender.1	0.30639	37.669	0.59868	1.26962
## nodeifactor.gender.1	0.37232	37.670	0.59869	1.27141
## nodeocov.alcohol	6.33721	116.779	1.85597	3.97827
## nodeicov.alcohol	6.54584	122.325	1.94411	4.12807
## absdiff.alcohol	-0.26623	50.475	0.80220	1.57395

##

2. Quantiles for each variable:

##

	2.5%	25%	50%	75%	97.5%
--	------	-----	-----	-----	-------


```

## edges -84.000 -31.000 0.0000 33.000 95.000
## nodematch.gender -83.000 -30.000 0.0000 34.000 93.000
## mutual -32.000 -11.000 0.0000 13.000 35.050
## gwesp.OTP.fixed.0.1 -100.557 -36.721 1.3579 40.279 114.761
## gwodeg.fixed.0.1 -5.247 -1.533 0.1990 1.781 4.279
## gwideg.fixed.0.1 -5.663 -1.702 0.2078 1.961 4.607
## nodeofactor.gender.1 -67.000 -27.000 -1.0000 25.000 78.000
## nodeifactor.gender.1 -67.000 -27.000 -1.0000 24.500 80.000
## nodeocov.alcohol -211.050 -76.000 2.0000 82.000 244.000
## nodeicov.alcohol -214.100 -78.000 3.0000 85.000 257.050
## absdiff.alcohol -94.000 -35.000 -2.0000 32.500 102.000
##
##
## Are sample statistics significantly different from observed?
## edges nodematch.gender mutual gwesp.OTP.fixed.0.1
## diff. 2.2793635 2.2841627 0.9191715 2.968131
## test stat. 1.3904276 1.4589449 1.5284688 1.579201
## P-val. 0.1643991 0.1445803 0.1263962 0.114290
## gwodeg.fixed.0.1 gwideg.fixed.0.1 nodeofactor.gender.1
## diff. 0.04214309 0.02784562 0.3063905
## test stat. 0.83697626 0.48881339 0.2413251
## P-val. 0.40260591 0.62497381 0.8093032
## nodeifactor.gender.1 nodeocov.alcohol nodeicov.alcohol
## diff. 0.3723162 6.337206 6.545845
## test stat. 0.2928362 1.592956 1.585693
## P-val. 0.7696473 0.111170 0.112809
## absdiff.alcohol (Omni)
## diff. -0.2662288 NA
## test stat. -0.1691465 25.556528815
## P-val. 0.8656814 0.008052915
##
## Sample statistics cross-correlations:
## edges nodematch.gender mutual gwesp.OTP.fixed.0.1
## edges 1.0000000 0.9757230 0.9317709 0.9785293
## nodematch.gender 0.9757230 1.0000000 0.9258228 0.9679443
## mutual 0.9317709 0.9258228 1.0000000 0.9492862
## gwesp.OTP.fixed.0.1 0.9785293 0.9679443 0.9492862 1.0000000
## gwodeg.fixed.0.1 0.4708714 0.4498849 0.4483660 0.4266192
## gwideg.fixed.0.1 0.4648140 0.4462667 0.4432268 0.4142682
## nodeofactor.gender.1 0.8079867 0.7910742 0.7491889 0.7918747
## nodeifactor.gender.1 0.8032780 0.7862700 0.7437564 0.7878244
## nodeocov.alcohol 0.9591899 0.9360376 0.9016265 0.9416844
## nodeicov.alcohol 0.9589828 0.9346047 0.8913238 0.9387741
## absdiff.alcohol 0.8244945 0.8000643 0.7592274 0.7988675
## gwodeg.fixed.0.1 gwideg.fixed.0.1 nodeofactor.gender.1
## edges 0.4708714 0.4648140 0.8079867
## nodematch.gender 0.4498849 0.4462667 0.7910742
## mutual 0.4483660 0.4432268 0.7491889
## gwesp.OTP.fixed.0.1 0.4266192 0.4142682 0.7918747
## gwodeg.fixed.0.1 1.0000000 0.3386004 0.3433523
## gwideg.fixed.0.1 0.3386004 1.0000000 0.3411299
## nodeofactor.gender.1 0.3433523 0.3411299 1.0000000
## nodeifactor.gender.1 0.3418181 0.3348452 0.9838166
## nodeocov.alcohol 0.4740957 0.4333060 0.6935012

```

```

## nodeicov.alcohol          0.4713878          0.4260358          0.6956318
## absdiff.alcohol          0.4231015          0.4100971          0.5934806
## nodeifactor.gender.1 nodeocov.alcohol nodeicov.alcohol
## edges                    0.8032780          0.9591899          0.9589828
## nodematch.gender         0.7862700          0.9360376          0.9346047
## mutual                   0.7437564          0.9016265          0.8913238
## gwesp.OTP.fixed.0.1      0.7878244          0.9416844          0.9387741
## gwodeg.fixed.0.1         0.3418181          0.4740957          0.4713878
## gwideg.fixed.0.1         0.3348452          0.4333060          0.4260358
## nodeofactor.gender.1     0.9838166          0.6935012          0.6956318
## nodeifactor.gender.1     1.0000000          0.6904251          0.6871942
## nodeocov.alcohol         0.6904251          1.0000000          0.9858228
## nodeicov.alcohol         0.6871942          0.9858228          1.0000000
## absdiff.alcohol          0.5893233          0.8225123          0.8360954
## absdiff.alcohol
## edges                    0.8244945
## nodematch.gender         0.8000643
## mutual                   0.7592274
## gwesp.OTP.fixed.0.1      0.7988675
## gwodeg.fixed.0.1         0.4231015
## gwideg.fixed.0.1         0.4100971
## nodeofactor.gender.1     0.5934806
## nodeifactor.gender.1     0.5893233
## nodeocov.alcohol         0.8225123
## nodeicov.alcohol         0.8360954
## absdiff.alcohol          1.0000000
##
## Sample statistics auto-correlation:
## Chain 1
## edges nodematch.gender mutual gwesp.OTP.fixed.0.1
## Lag 0 1.0000000 1.0000000 1.0000000 1.0000000
## Lag 24000 0.6202566 0.6435884 0.6289399 0.6386142
## Lag 48000 0.4116515 0.4228933 0.4054584 0.4197581
## Lag 72000 0.2867670 0.2873365 0.2813267 0.2894669
## Lag 96000 0.2025114 0.2028105 0.1939781 0.2016381
## Lag 120000 0.1355932 0.1311716 0.1288809 0.1262960
## gwodeg.fixed.0.1 gwideg.fixed.0.1 nodeofactor.gender.1
## Lag 0 1.0000000 1.0000000 1.0000000
## Lag 24000 0.13820566 0.13443423 0.6361161
## Lag 48000 0.08097046 0.08423023 0.4178025
## Lag 72000 0.04893458 0.08089974 0.2817434
## Lag 96000 0.02195496 0.06069668 0.1961316
## Lag 120000 0.04307426 0.02231194 0.1336774
## nodeifactor.gender.1 nodeocov.alcohol nodeicov.alcohol
## Lag 0 1.0000000 1.0000000 1.0000000
## Lag 24000 0.6369396 0.6133477 0.5986978
## Lag 48000 0.4178373 0.4060623 0.3979328
## Lag 72000 0.2824987 0.2792334 0.2747402
## Lag 96000 0.1989718 0.1969080 0.1917472
## Lag 120000 0.1380993 0.1318763 0.1286975
## absdiff.alcohol
## Lag 0 1.0000000
## Lag 24000 0.4868468
## Lag 48000 0.2910116

```

```

## Lag 72000          0.1863064
## Lag 96000          0.1473820
## Lag 120000         0.1101644
##
## Sample statistics burn-in diagnostic (Geweke):
## Chain 1
##
## Fraction in 1st window = 0.1
## Fraction in 2nd window = 0.5
##
##          edges      nodematch.gender      mutual
##      -0.15138298      -0.14588321      -0.06055932
## gwesp.OTP.fixed.0.1 gwodeg.fixed.0.1 gwideg.fixed.0.1
##      -0.08622441      0.41516841      0.76217049
## nodeofactor.gender.1 nodeifactor.gender.1 nodeocov.alcohol
##      0.65095168      0.55865531      -0.10720981
##      nodeicov.alcohol      absdiff.alcohol
##      -0.25959011      -0.94812182
##
## Individual P-values (lower = worse):
##          edges      nodematch.gender      mutual
##      0.8796736      0.8840136      0.9517102
## gwesp.OTP.fixed.0.1 gwodeg.fixed.0.1 gwideg.fixed.0.1
##      0.9312880      0.6780186      0.4459583
## nodeofactor.gender.1 nodeifactor.gender.1 nodeocov.alcohol
##      0.5150777      0.5763970      0.9146225
##      nodeicov.alcohol      absdiff.alcohol
##      0.7951800      0.3430674
## Joint P-value (lower = worse): 0.2120706
##
## Note: To save space, only one in every 12 iterations of the MCMC sample
## used for estimation was stored for diagnostics. Sample size per chain
## was originally around 47508 with thinning interval 2000.
##
## Note: MCMC diagnostics shown here are from the last round of
## simulation, prior to computation of final parameter estimates.
## Because the final estimates are refinements of those used for this
## simulation run, these diagnostics may understate model performance.
## To directly assess the performance of the final model on in-model
## statistics, please use the GOF command: gof(ergmFitObject,
## GOF=~model).

```

2.3

The ERGM model here itself has the 7 parameters to check for the wanted features (*i* - *vii*). To achieve convergence we had to add in 2 more control parameters: *gwodegree* and *gwidegree*. We see in the summary, that both *gwodegree* and *gwidegree* are significant, so they were important to explain the network and help with the convergence of the algorithm. Using a high decay value like $\alpha = 0.3$ or $\alpha = 0.6$ gave problems while converging so we went for one on the lower bound of $\alpha = 0.1$, since the MCMC sampler tended to generate networks with way more edges than observed. The low α decay value also tells that in the network, transitivity is only important over short distances, and longer chains do not contribute much. Using small parameters in the control parameter, the *mcmc.diagnostics()* gave some bad results: The Mean was far away from 0 for many terms, the P-value was below or very close to 0.05 and the omnibus value being at

3.4×10^{-6} indicates the model has not fully converged yet. All of this indicated that a longer MCMC run with larger parameters was needed as it is now the case. So from this first model we now expand on it further by changing the parameters in control to larger samplesize, burnin and interval. Further all this is done on the initial model as it just has not fully converged yet, and starting from these large settings gave a failure of convergence, as these are bad starting parameters.

This second ERGM fit may have only one iteration until it finishes, but it improves most statistics shown in `mcmc.diagnostics()`. All terms now have a P-value above 0.05, but most are still not close to 1. All individual Geweke statistics are within acceptable bounds and the Geweke joint P-value improved from 0.089 to 0.212, confirming chain stationarity. The omnibus P-value is significantly bigger at 0.008 >> 3.4×10^{-6} but still below 0.05, which is fine since it is sensitive to large sample sizes and not indicative of systematic misfit in any specific statistic.

2.4

```
gof_model <- gof(bigger_model, GOF = ~ idegree + odegree + espartners + distance) ## dspartners
saveRDS(gof_model, "gof_model.rds")
```

```
gof_model <- readRDS("gof_model.rds")
# Plot auxiliary statistics
gof_model
```

```
##
## Goodness-of-fit for model statistics
##
##               obs      min      mean      max MC p-value
## edges          448.0000 345.0000 447.1000 554.0000      0.94
## nodematch.gender 394.0000 281.0000 393.6400 504.0000      0.88
## mutual          124.0000  84.0000 125.8500 163.0000      0.92
## gwesp.OTP.fixed.0.1 325.1830 200.0814 325.3779 460.9449      0.98
## gwodeg.fixed.0.1  134.9205 125.6260 134.8538 140.3149      0.96
## gwideg.fixed.0.1  133.7391 125.7896 133.8034 140.1647      1.00
## nodeofactor.gender.1 266.0000 181.0000 265.2500 360.0000      1.00
## nodeifactor.gender.1 266.0000 185.0000 265.7900 358.0000      0.98
## nodeocov.alcohol 1084.0000 787.0000 1087.0400 1377.0000      1.00
## nodeicov.alcohol 1136.0000 816.0000 1139.7600 1461.0000      0.92
## absdiff.alcohol   446.0000 334.0000 444.7400 567.0000      0.94
##
## Goodness-of-fit for minimum geodesic distance
##
##      obs  min   mean  max MC p-value
## 1    448  345  447.10  554      0.94
## 2    777  834 1312.68 3677      0.00
## 3   1162 1424 2562.32 4118      0.00
## 4   1580 1737 3365.75 4564      0.00
## 5   1802 1818 3013.10 3770      0.00
## 6   1739  915 1927.51 2721      0.54
## 7   1625  164  986.96 1656      0.04
## 8   1274    8  455.64 1326      0.02
## 9    872    0  203.26  933      0.02
## 10   573    0   89.62  557      0.00
## 11   287    0   38.04  270      0.00
```

```

## 12    103    0    15.31  132    0.10
## 13     21    0     6.09   73    0.20
## 14      5    0     2.36   35    0.24
## 15      0    0     0.76   21    1.00
## 16      0    0     0.24    9    1.00
## 17      0    0     0.04    2    1.00
## Inf 4244  632 2085.22 4353    0.02
##
## Goodness-of-fit for out-degree
##
##      obs min  mean max MC p-value
## 0      6   0  4.48  12    0.56
## 1      7  12 23.48  41    0.00
## 2     25  14 26.88  39    0.80
## 3     28  10 23.05  34    0.36
## 4     25  10 16.90  26    0.08
## 5     21   4 12.01  22    0.04
## 6     17   1  7.94  15    0.00
## 7      0   1  5.16  14    0.00
## 8      0   0  3.18   8    0.06
## 9      0   0  2.27   8    0.22
## 10     0   0  1.28   5    0.66
## 11     0   0  0.88   4    0.78
## 12     0   0  0.53   3    1.00
## 13     0   0  0.34   4    1.00
## 14     0   0  0.16   2    1.00
## 15     0   0  0.16   1    1.00
## 16     0   0  0.14   2    1.00
## 17     0   0  0.07   2    1.00
## 18     0   0  0.05   1    1.00
## 19     0   0  0.01   1    1.00
## 22     0   0  0.01   1    1.00
## 23     0   0  0.01   1    1.00
## 56     0   0  0.01   1    1.00
##
## Goodness-of-fit for in-degree
##
##      obs min  mean max MC p-value
## 0      6   0  5.45  12    0.90
## 1     18  11 23.36  34    0.38
## 2     28  16 26.04  38    0.72
## 3     23  12 22.46  33    1.00
## 4     18   9 17.34  25    1.00
## 5     12   4 12.02  21    1.00
## 6      9   2  7.67  14    0.78
## 7      8   1  5.48  12    0.42
## 8      2   0  3.18   9    0.76
## 9      4   0  2.14   6    0.34
## 10     0   0  1.46   5    0.46
## 11     1   0  0.85   4    1.00
## 12     0   0  0.70   3    1.00
## 13     0   0  0.31   2    1.00
## 14     0   0  0.13   1    1.00
## 15     0   0  0.12   2    1.00

```

```

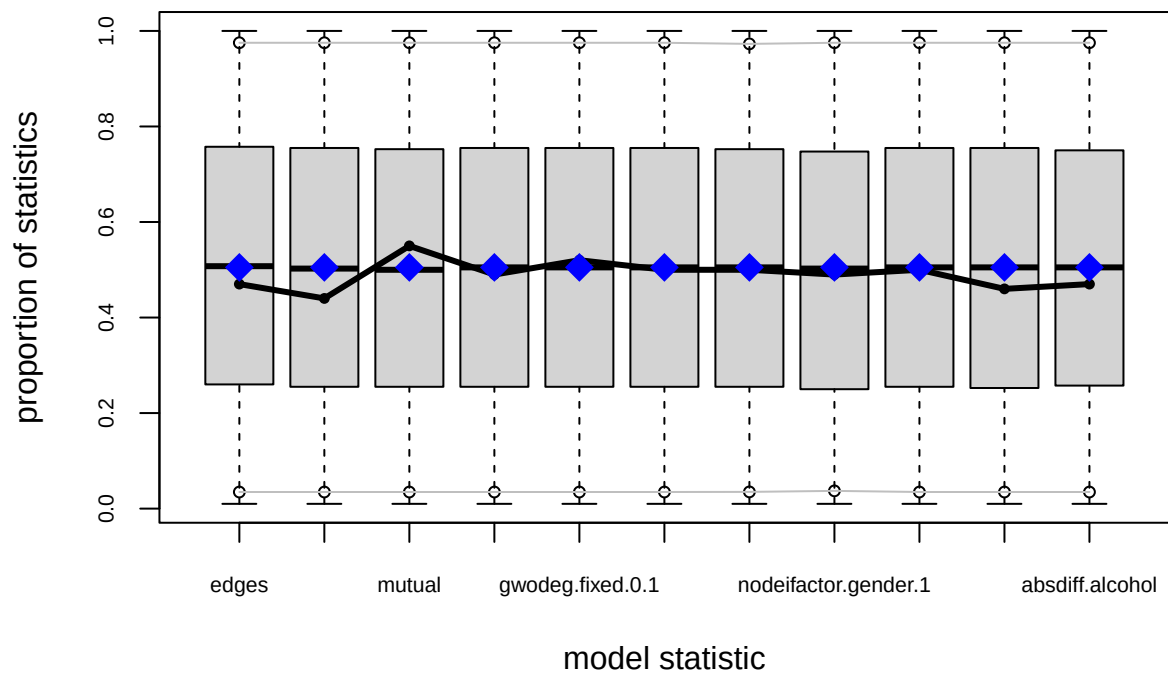
## 16  0  0  0.08  1      1.00
## 17  0  0  0.08  1      1.00
## 18  0  0  0.04  1      1.00
## 19  0  0  0.01  1      1.00
## 20  0  0  0.02  1      1.00
## 22  0  0  0.01  1      1.00
## 23  0  0  0.02  1      1.00
## 24  0  0  0.01  1      1.00
## 26  0  0  0.01  1      1.00
## 61  0  0  0.01  1      1.00
##
## Goodness-of-fit for edgewise shared partner
##
##      obs min   mean max MC p-value
## .0TP0 137  92 130.07 162    0.50
## .0TP1 167 150 231.11 304    0.06
## .0TP2  92  28  67.39 110    0.28
## .0TP3  42   3  14.30  36    0.00
## .0TP4   9   0   3.26  17    0.18
## .0TP5   1   0   0.70  11    0.70
## .0TP6   0   0   0.22   5    1.00
## .0TP7   0   0   0.04   3    1.00
## .0TP9   0   0   0.01   1    1.00

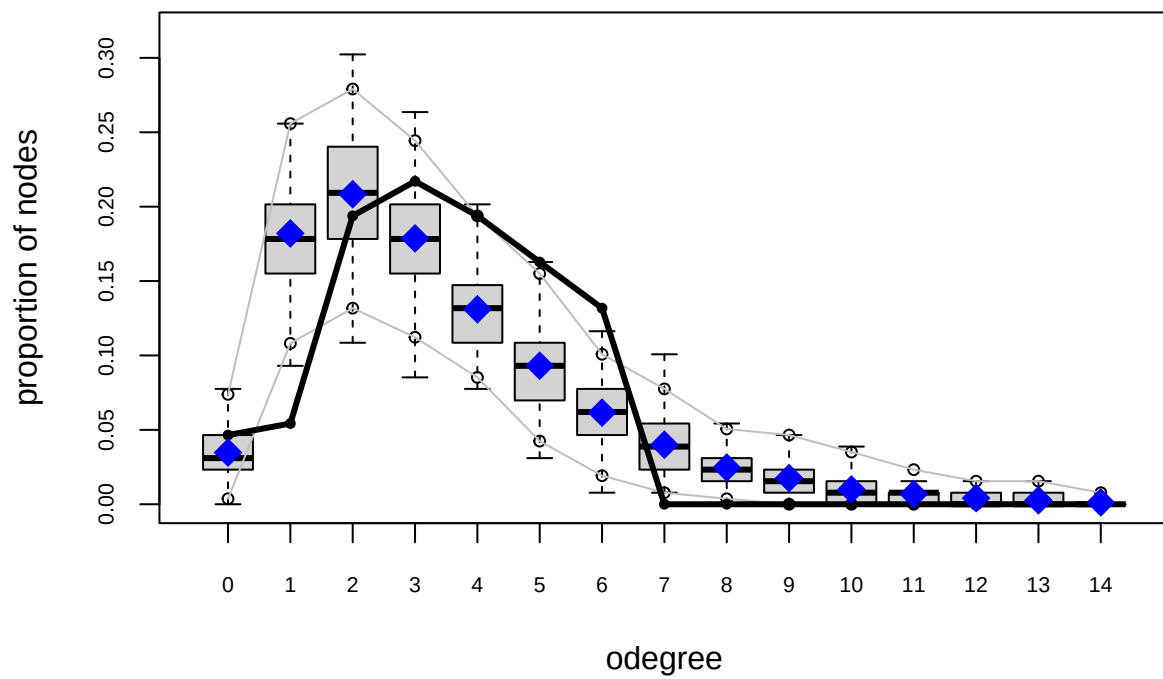
```

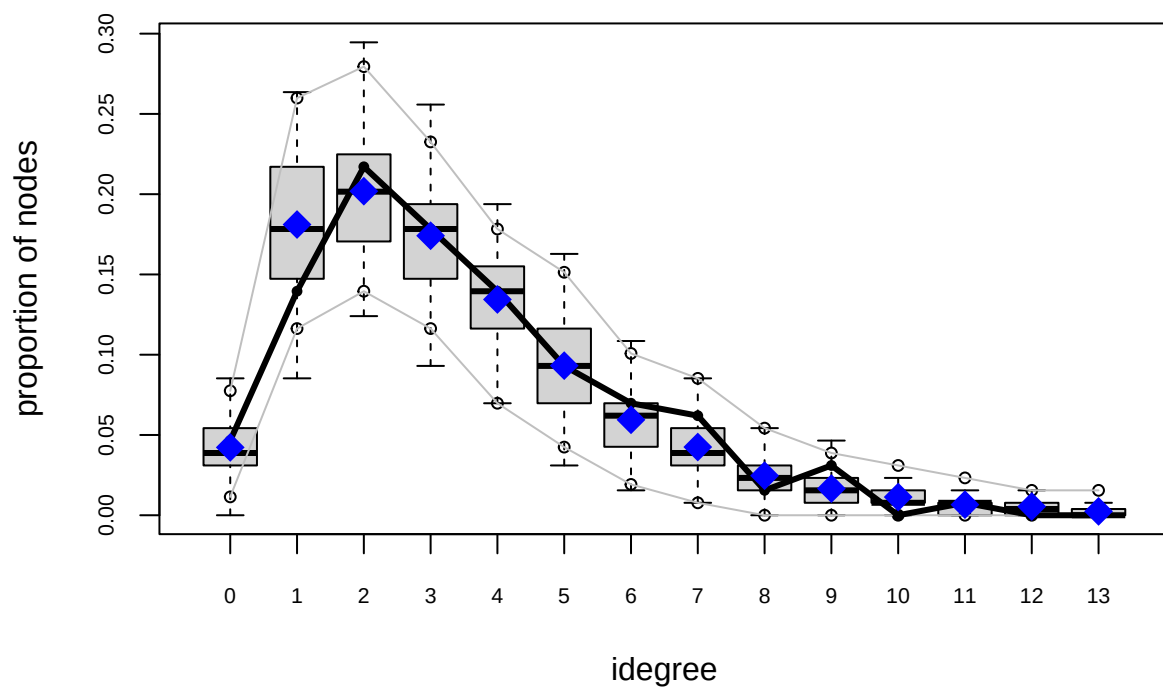
```

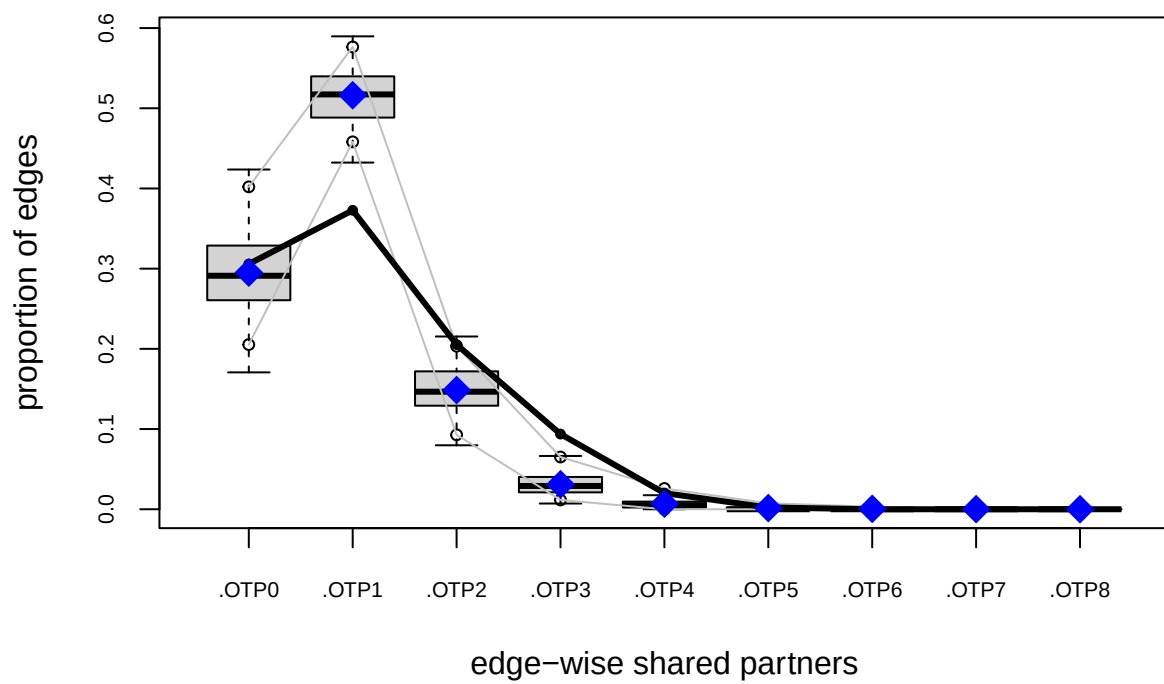
ergm::plot.gof(gof_model)

```

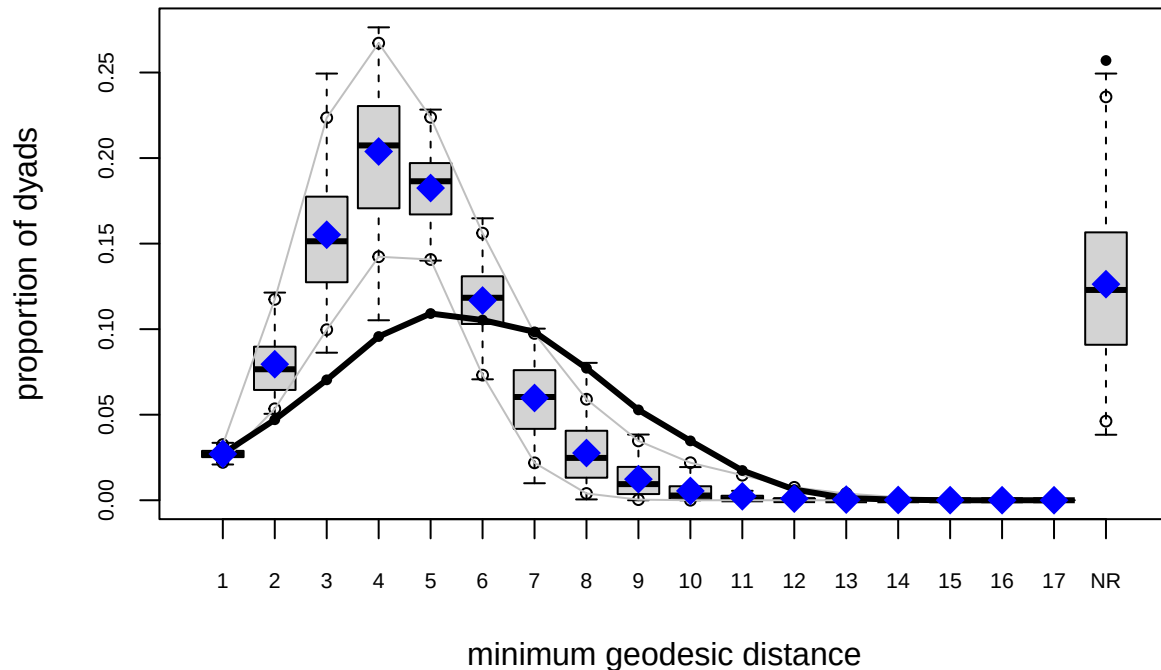








Goodness-of-fit diagnostics



Both the plot of *model statistic* and *idegree* indicate a very good fit on these terms, as there are no outliers and their values are very close to the centre of the boxplots. This confirms that the model is internally consistent and that it reproduces the in-degree distribution well, it accurately captures how many nominations students receive. The *odegree* plot is somewhat predictably wrong, it overestimates the count of nodes that have exactly 1 outgoing edge and then has too few with those that have 3-6 edges. It wrongly assumes that there are some nodes with more than 6 outgoing edges, which is not possible per the data collection, as that only allowed up to 6 friendship nominations per person, which is information that the model does not know about. The *edge-wise shared partners* plot shows that the model has problems with predicting triads, which suggests that the chosen decay value $\alpha = 0.1$ might not have been the best choice. The bad fitting of the *minimum geodesic distance* plot shows that the estimated model is more fragmented and less connected than the real observed data would suggest. Overall, the model shows clear parts that can still be improved, for one there is the α decay value that could be tweaked until there are better results. There could also be more parameters that the model does not count for yet, like the fact that a node has a maximum out-degree of 6. It seems to be better with replicating local features, while its weaknesses lie with the global features.

2.5

```
summary(bigger_model)
```

```
## Call:
## ergm(formula = net ~ edges + nodematch("gender") + mutual + gwesp(0.1,
##   fixed = TRUE) + gwodegree(0.1, fixed = TRUE) + gwidegree(0.1,
##   fixed = TRUE) + nodeofactor("gender", base = 2) + nodeifactor("gender",
```

```

##      base = 2) + nodecov("alcohol") + nodeicov("alcohol") + absdiff("alcohol"),
##      control = control.ergm(seed = 161, MCMC.samplesize = 50000,
##          init = coef(big_model), MCMC.burnin = 1e+05, MCMC.interval = 2000,
##          MCMLE.maxit = 60))
##
## Monte Carlo Maximum Likelihood Results:
##
##              Estimate Std. Error MCMC % z value Pr(>|z|)
## edges          -5.95141    0.17513     0 -33.983 < 1e-04 ***
## nodematch.gender    0.85499    0.10496     0  8.146 < 1e-04 ***
## mutual           3.02297    0.19107     0 15.821 < 1e-04 ***
## gwesp.OTP.fixed.0.1  1.67667    0.10814     0 15.505 < 1e-04 ***
## gwodeg.fixed.0.1    1.84220    0.48991     0  3.760 0.000170 ***
## gwideg.fixed.0.1    1.60488    0.44954     0  3.570 0.000357 ***
## nodeofactor.gender.1 -0.10194    0.15031     0 -0.678 0.497628
## nodeifactor.gender.1  0.03781    0.14949     0  0.253 0.800318
## nodecov.alcohol     -0.11891    0.05540     0 -2.147 0.031823 *
## nodeicov.alcohol     0.15648    0.05472     0  2.860 0.004242 **
## absdiff.alcohol     -0.11072    0.03710     0 -2.985 0.002840 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 22890 on 16512 degrees of freedom
##      Residual Deviance: 2845 on 16501 degrees of freedom
##
##      AIC: 2867 BIC: 2952 (Smaller is better. MC Std. Err. = 0.6422)

```

Terms not used for any hypothesis: The *edges* coefficient of $(-5.95, p < 0.001)$ confirms that the friendship network is very sparse and a the probability for a tie existing between any two students is very low. Very logical for a huge network, where any student can only nominate a maximum of 6 friends. Both *gwodegree* $(+1.84, p < 0.001)$ and *gwidegree* $(+1.60, p < 0.001)$ are significant, confirming there is a clear difference between how many nominations students both send and receive. Some students are far more social or popular than others. The gender homophily from 2.1 still stays relevant here with *nodematch.gender* $(+0.85, p < 0.001)$ and indicates that it plays an important role in how ties are formed. It is to be noted though, that this term dropped from the model in 2.1 with a value of $+2.01$, so some of the networks model can be better explained by other terms introduced in this new bigger model.

Hypothesis (i) Reciprocity: *mutual* $(+3.02, p < 0.001)$. The hypothesis is strongly supported by these values. If student j has nominated student i , the odds of i nominating j back are multiplied by $e^{3.02} \approx 20.5$. This is the strongest effect that is found in this model, indicating it is a crucial deciding factor in how nominations are built up on.

Hypothesis (ii) Transitivity: *gwesp* $(+1.68, p < 0.001)$. The hypothesis is strongly supported by these values. When two students share a common friend, a tie between them becomes significantly more likely. This supports a typical triadic closure structure in the model, which occurs in most social networks and also leads to the formation of clusters.

Hypothesis (iii) Boys have higher social activity: *nodeofactor.gender* $(-0.10, p = 0.498)$. This hypothesis is not supported by these values. The coefficient for boys sending more nominations than girls is small and negative, which would suggest that the opposite is slightly true. This however is not the case either, since the P-value is so high and as such, not making it statistically significant.

Hypothesis (iv) Boys have higher popularity: *nodeifactor.gender* $(+0.04, p = 0.800)$. This hypothesis is not supported by these values. The coefficient is near zero and the high P-value means it is completely insignificant. Boys do not receive significantly more nominations than girls in this network, which, along

with Hypothesis (iii), means that gender has no direct influence on how popular or socially active a person is.

Hypothesis (v) Alcohol consumption increases social activity: *nodecov.alcohol* ($-0.12, p = 0.032$). This hypothesis is not supported by these values, the somewhat low P-value along a negative coefficient say the opposite is supported. Higher alcohol consumption is associated with fewer outgoing nominations. A one unit increase in alcohol score decreases the odds-ratio of sending a tie by $e^{0.12} \approx 1.127$, suggesting that heavier drinkers are actually less likely to actively nominate friends, contrary to the hypothesis. With four units difference from 1 to 5, this means an approximate odd-ratio decrease of $e^{0.12*4} \approx 1.616$ for the heaviest drinkers in comparison to those with a value of 1.

Hypothesis (vi) Alcohol consumption increases popularity: *nodecov.alcohol* ($+0.16, p = 0.004$). The hypothesis is supported by these values. Students with higher alcohol consumption receive significantly more friendship nominations. A one unit increase in alcohol score multiplies the odds of receiving a tie by $e^{0.16} \approx 1.174$, suggesting heavier drinkers are perceived as more popular or socially attractive by their peers. With four units difference from 1 to 5, this means an approximate odd increase of $e^{0.16*4} \approx 1.896$ for the heaviest drinkers in comparison to those with a value of 1.

Hypothesis (vii) Alcohol homophily: *absdiff.alcohol* ($-0.11, p = 0.003$). The hypothesis is supported by these values. The negative coefficient on absolute difference means ties are more likely between students with similar alcohol consumption levels. The smaller the difference in alcohol scores between two students, the more likely they are to form a friendship, indicating clear homophilic tendencies regarding alcohol consumption.

2.6

Since the data collection method only allowed the students to nominate 6 friends, a constraint that enforces this could be added to the ERGM estimation model to get this point correct, which would make the model itself more related to the model and fit some terms better. This would however raise the question if this is really wanted, since it is very possible that in real-life, students have more than 6 friendship nominations. So this should rather a problem to think about at the data collection part of data modelling. From the results we found in 2.4, we should tweak the α decay value to a higher value like 0.3 or maybe even higher than that, since transitivity between students friendships seems to hold over more nodes than the current value estimates for and it does not correctly simulate the triadic patterns that can be found in this network. There is a very arguable point to be made that ties are also dependent on age-homophily, something that the current model does not account for at all, since the edges do not even have the age value added to them. There is really no reason not to add the age parameter to the model, since we already have the data available. Adding these terms:

```
# + nodecov("age") + nodecov("age") + absdiff("age")
```

could help the model. Statistical outliers, like people who have to repeat a class, may be on one end of the spectrum and more likely to have drastically different data values than other students.

2.7

We suggest two hypotheses that center around the uncollected data of the students grades. The first hypothesis is centered around Academic homophily. This would require the following addition to the model:

```
# + absdiff("grade")
```

This could be computed like this: $\text{absdiff}(\text{"grade"}) = \sum_{i \neq j} y_{ij} \cdot |x_i^{\text{grade}} - x_j^{\text{grade}}|$, where $y_{ij} = 1$ if student i has nominated student j and is 0 otherwise. x_k^{grade} corresponds to the grade of student k . A negative coefficient

would support the hypothesis in this case, as it would make ties more probable if they have similar grade levels. In the network graph, this effect would be seen as more ties existing between students of similar academic level. It should also be noted, that since we just assume one total grade for each student, there might be different results if the same hypothesis is executed on just single subject grades.

The second hypothesis is that students with higher grades receive more friendship nominations. That would require this addition to the model to be figured out if the hypothesis is supported or not:

```
# + nodeicov("grade")
```

This could be computed like this: $\text{nodeicov}(\text{"grade"}) = \sum_{i \neq j} y_{ij} \cdot x_j^{\text{grade}}$. A positive coefficient would support this hypothesis, as that would mean that students with higher grades receive more nominations. In the network graph, this could be seen as nodes with a high grade value having a relatively higher indegree than those with a lower grade.

Both of these hypotheses could explain some residual variation in tie formation, that has not yet been captured by our current model.

Task 3: Estimation and interpretation of SAOMs

3.1

3.1.1

```
install.packages("RSiena")
```

```
library(RSiena)
```

```
## Registered S3 method overwritten by 'RSiena':
```

```
##   method      from
```

```
##   plot.sienaGOF autograph
```

```
##
```

```
## Attache Paket: 'RSiena'
```

```
## Das folgende Objekt ist maskiert 'package:xtable':
```

```
##
```

```
##      xtable
```

```
library(parallel)
```

```
library(sna)
```

```
# Load friendship waves
```

```
f1 <- matrix(as.numeric(as.matrix(read_matrix("Glasgow/f1.csv", header = FALSE))),
             nrow = 129, ncol = 129)
```

```
f2 <- matrix(as.numeric(as.matrix(read_matrix("Glasgow/f2.csv", header = FALSE))),
             nrow = 129, ncol = 129)
```

```
f3 <- matrix(as.numeric(as.matrix(read_matrix("Glasgow/f3.csv", header = FALSE))),
             nrow = 129, ncol = 129)
```

```

# Read demographic, alcohol and logdistance data
demographic <- read.csv("Glasgow/demographic.csv", header = TRUE)
alcohol_data <- read.csv("Glasgow/alcohol.csv", header = TRUE)
logdist <- matrix(as.numeric(unlist(strsplit(readLines("Glasgow/logdistance.csv"), ","))), nrow = 129, ncol = 3)

friendship <- as_dependent_rsiena(array(c(f1, f2, f3), dim = c(129, 129, 3)), allowOnly = FALSE)

# Create covariates
gender <- as_covariate_rsiena(demographic$gender, type = "monadic")
alcohol <- as_covariate_rsiena(alcohol_data[, 1], type = "monadic")
logdistance <- as_covariate_rsiena(logdist, type = "oneMode")

# Combine into siena data object
mydata <- make_data_rsiena(friendship, gender, alcohol, logdistance)
#print(mydata)

# Compute Jaccard index and data description
write_report(mydata, outputName = "glasgowJaccardIndex")

```

From the data generated in the file “glasgowJaccardIndex”, we have the resulting Jaccard Index values from Wave 1 → Wave 2 with a value of 0.304 and from Wave 2 → Wave 3 with a value of 0.351. Both of these values fit neatly into the recommended range of [0.2, 0.6], indicating that the data contains sufficient data to estimate a SAOM. If the value were below that range, it would mean the network changes too drastically and leaves no network stability, while a value above the range would indicate that the network hardly changes at all and shows no evolutionary traits. The higher Jaccard index from Wave 2 → Wave 3 indicates that the network gained stability over time.

3.1.2

```

myeff <- make_specification(mydata)

#endogenous structural effects, from what generally should hold for networks
myeff <- set_effect(myeff, list(recip)) # reciprocity

##   effectNumber effectName  shortName include fix   test  initialValue parm
## 1 17            reciprocity recip      TRUE  FALSE FALSE           0    0

myeff <- set_effect(myeff, list(transTrip)) # transitivity

##   effectNumber effectName          shortName include fix   test  initialValue
## 1 23            transitive triplets transTrip TRUE  FALSE FALSE           0
##   parm
## 1 0

myeff <- set_effect(myeff, list(cycle3)) # 3-cycles

##   effectNumber effectName shortName include fix   test  initialValue parm
## 1 43            3-cycles  cycle3      TRUE  FALSE FALSE           0    0

```

```
#exogenous control effects, we know from task 2 that gender plays a huge role
myeff <- set_effect(myeff, list(egoX, altX, sameX), covar1 = "gender")
```

```
##   effectNumber effectName  shortName include fix   test  initialValue parm
## 1 297          gender alter altX      TRUE  FALSE FALSE         0    0
## 2 312          gender ego  egoX      TRUE  FALSE FALSE         0    0
## 3 368          same gender sameX      TRUE  FALSE FALSE         0    0
```

```
#Hypothesis i: popularity
```

```
myeff <- set_effect(myeff, list(inPopSqrt))
```

```
##   effectNumber effectName          shortName include fix   test
## 1 86          indegree - popularity (sqrt) inPopSqrt TRUE  FALSE FALSE
##   initialValue parm
## 1          0    0
```

```
#Hypothesis ii: alcohol homophily
```

```
myeff <- set_effect(myeff, list(egoX, altX, simX), covar1 = "alcohol")
```

```
##   effectNumber effectName          shortName include fix   test  initialValue
## 1 481          alcohol alter      altX      TRUE  FALSE FALSE         0
## 2 496          alcohol ego        egoX      TRUE  FALSE FALSE         0
## 3 542          alcohol similarity simX      TRUE  FALSE FALSE         0
##   parm
## 1 0
## 2 0
## 3 0
```

```
#Hypothesis iii: geographic proximity
```

```
myeff <- set_effect(myeff, list(X), covar1 = "logdistance")
```

```
##   effectNumber effectName  shortName include fix   test  initialValue parm
## 1 264          logdistance X      TRUE  FALSE FALSE         0    0
```

```
myeff
```

```
##   effectName          include fix   test  initialValue parm
## 1 constant friendship rate (period 1) TRUE  FALSE FALSE  7.45424  0
## 2 constant friendship rate (period 2) TRUE  FALSE FALSE  6.82927  0
## 3 outdegree (density)      TRUE  FALSE FALSE -1.61299  0
## 4 reciprocity             TRUE  FALSE FALSE  0.00000  0
## 5 transitive triplets      TRUE  FALSE FALSE  0.00000  0
## 6 3-cycles                 TRUE  FALSE FALSE  0.00000  0
## 7 indegree - popularity (sqrt) TRUE  FALSE FALSE  0.00000  0
## 8 logdistance              TRUE  FALSE FALSE  0.00000  0
## 9 gender alter             TRUE  FALSE FALSE  0.00000  0
## 10 gender ego              TRUE  FALSE FALSE  0.00000  0
## 11 same gender             TRUE  FALSE FALSE  0.00000  0
## 12 alcohol alter           TRUE  FALSE FALSE  0.00000  0
## 13 alcohol ego             TRUE  FALSE FALSE  0.00000  0
## 14 alcohol similarity      TRUE  FALSE FALSE  0.00000  0
```

3.1.3

```
#use different algorithm package thingy to allow constraint.
SAOMAlgorithm <- sienaAlgorithmCreate(seed = 161, n3 = 3000, nsub = 4, MaxDegree = c(friendship = 6))
```

```
## If you use this algorithm object, siena07 will create/use an output file Siena.txt .
```

```
# Estimate the model
```

```
model_saom <- siena07(SAOMAlgorithm,
  data = mydata,
  effects = myeff,
  returnDeps = TRUE,
  batch = FALSE,
)
saveRDS(model_saom, "model_saom.rds")
```

```
model_saom <- readRDS("model_saom.rds")
#Check results and convergence criterion (as presented in the lecture slides)
#model_saom
t_ratios <- apply(model_saom$sf, 2, mean) / apply(model_saom$sf, 2, sd)
t_ratios
```

```
## [1] -0.048280819 -0.046630219 -0.044989376 -0.055399050 -0.000493128
## [6]  0.036705480 -0.047920270 -0.016010575
```

```
overall <- sqrt(t(apply(model_saom$sf, 2, mean)) %*% solve(cov(model_saom$sf)) %*% apply(model_saom$sf,
overall
```

```
##           [,1]
## [1,] 0.1158141
```

As we can see from these results, we have that all absolute values of the t-ratios are below 0.1, along with the overall value being below 0.25. This fulfills the convergence criterion, indicating that all of our chosen parameters were sufficient to achieve convergence. The SAOM was estimated with 4 subphases and 3000 iterations in phase 3, which was enough to achieve this convergence and did not need to have another re-estimation or other model adjustments. These parameters consist of the three needed for answering our hypotheses: One parameter to control for the popularity of a student by checking for its indegree value, another to check for alcohol consumption homophily and the last one to see if geographic proximity has an affect on the model evolution. Furthermore we have some basic endogenous and exogenous variables that normally play a big part in the structure of most networks: Variables to check for the reciprocity of ties, the transitivity of ties and 3-cyclic patterns. Lastly we added in one more parameter, to give control for gender homophily. We have seen in task 2 that it plays a huge role in the structure of this specific network.

3.1.4


```

# Geodesic distance
GeodesicDistribution <- function(
  i, data, sims, period, groupName,
  varName, levls = c(1:5, Inf), cumulative = TRUE
) {
  x <- networkExtraction(i, data, sims, period, groupName, varName)
  require(sna)
  a <- sna::geodist(symmetrize(x))$gdist
  if (cumulative) {
    gdi <- sapply(levls, function(i) { sum(a <= i) })
  } else {
    gdi <- sapply(levls, function(i) { sum(a == i) })
  }
  names(gdi) <- as.character(levls)
  gdi
}

```

```

#most of this is just from the practical:
cl <- makeCluster(4)

# Indegree distribution
gof_id <- test_gof(
  model_saom,
  verbose = FALSE,
  varName = "friendship", IndegreeDistribution,
  cluster = cl
)

# Outdegree distribution
gof_od <- test_gof(
  model_saom,
  verbose = FALSE,
  varName = "friendship", OutdegreeDistribution,
  cluster = cl
)

# Triad census
gof_tc <- test_gof(
  model_saom,
  verbose = FALSE,
  varName = "friendship", TriadCensus,
  cluster = cl
)

gof_gd <- test_gof(
  model_saom,
  verbose = FALSE,
  varName = "friendship", GeodesicDistribution
)

stopCluster(cl)
saveRDS(gof_id, "gof_id.rds")
saveRDS(gof_od, "gof_od.rds")

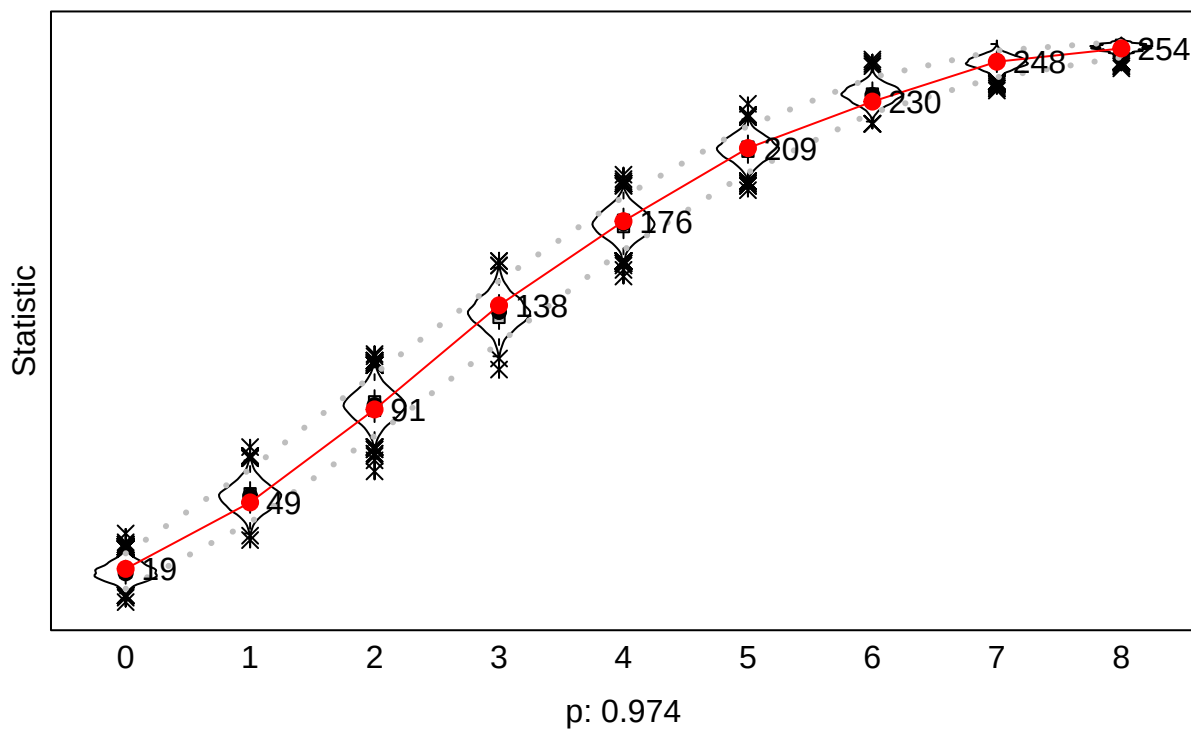
```

```
saveRDS(gof_gd, "gof_gd.rds")
saveRDS(gof_tc, "gof_tc.rds")
```

```
gof_id <- readRDS("gof_id.rds")
gof_od <- readRDS("gof_od.rds")
gof_gd <- readRDS("gof_gd.rds")
gof_tc <- readRDS("gof_tc.rds")
```

```
#Plot all four
plot(gof_id)
```

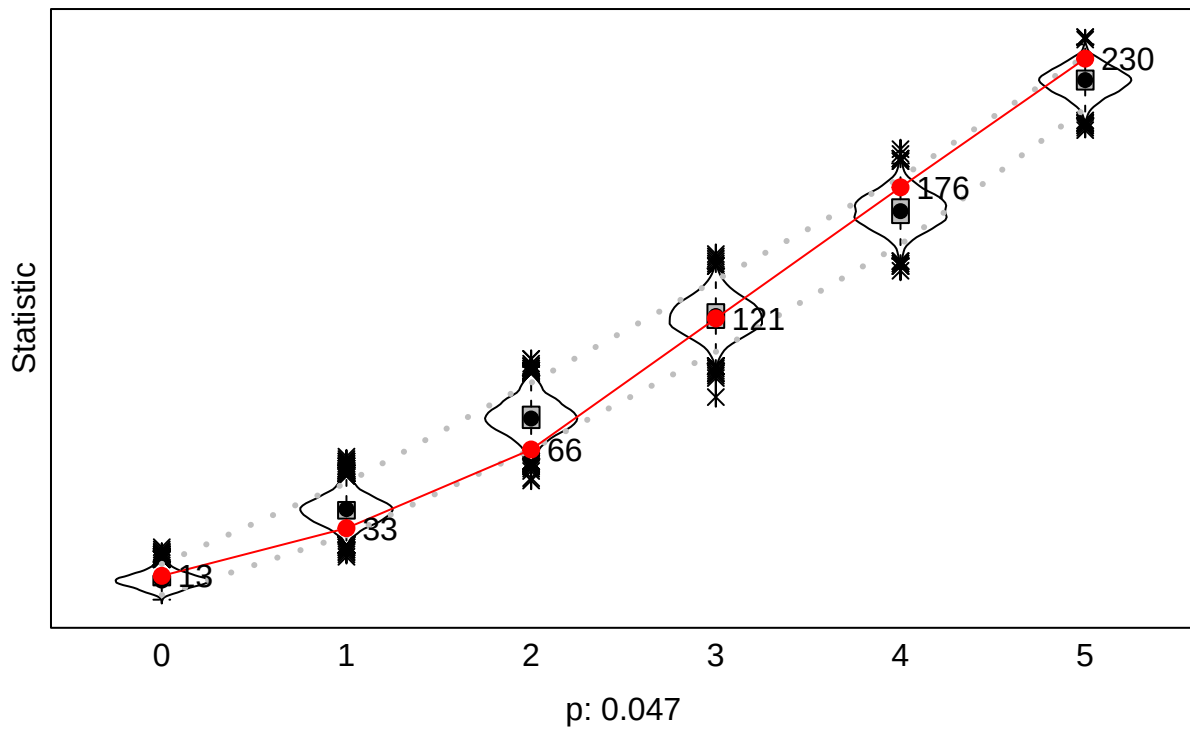
Goodness of Fit of IndegreeDistribution



```
plot(gof_od)
```

```
## Note: some statistics are not plotted because their variance is 0.
## This holds for the statistics: 6 7 8.
```

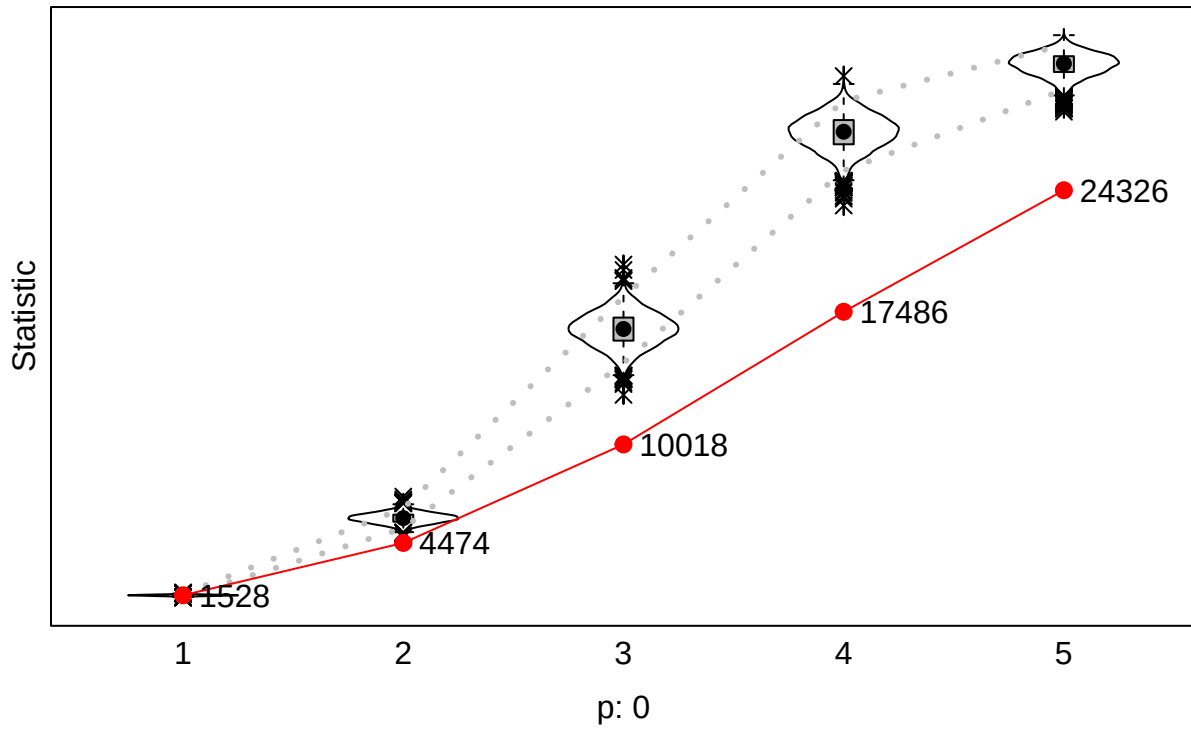
Goodness of Fit of OutdegreeDistribution



```
plot(gof_gd)
```

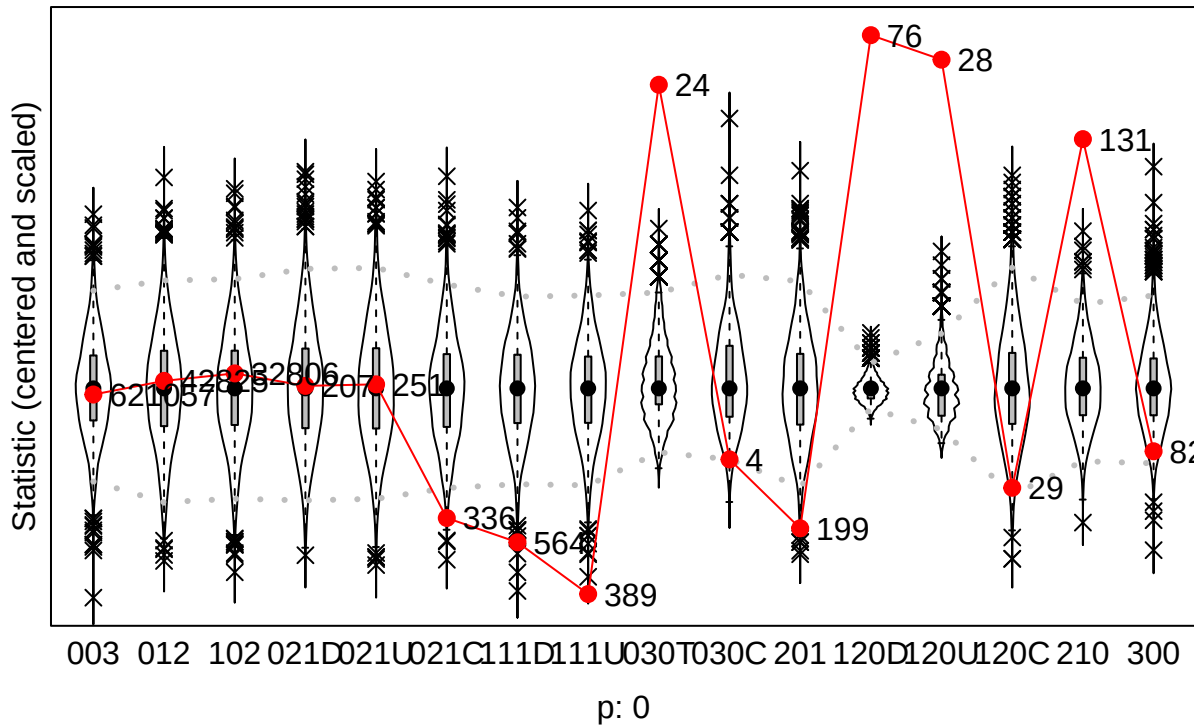
```
## Note: some statistics are not plotted because their variance is 0.  
## This holds for the statistic: Inf.
```

Goodness of Fit of GeodesicDistribution



```
plot(gof_tc, center = TRUE, scale = TRUE)
```

Goodness of Fit of TriadCensus



The plot for the goodness of fit for the indegree distribution shows that our SAOM model captures this property well. The plot follows the real data and as such produces a p-value of 0.974, which supports that the model captures this property well, since high p-values close to 1 support the property being captured. The Outdegree distribution is at a low p-value of 0.047, which means it somewhat captures this property. This had a p-value of 0 before implementing the maximum outdegree constraint of 6 per student, as the data collection method enforces, meaning that the constraint is very important to capture the outdegree structure of the observed network, but it seems there are still some other variables that are important for the development of the network that were not accounted for so far.

The other two plotted parameters are both not captured well. To start of with, they both have a p-value of 0, which means that neither of them accurately capture the observed data. The geodesic distribution itself has values that are higher than they are supposed to be over all distances, granting it a p-value of 0. This means that the simulated network is more connected than the real one, and that the real one has more of a community type structured one, meaning there are more separate clusters that have only few edges going out / coming in. The Triad consensus also has a p-value of 0, and as seen in the graph, while it correctly predicts some kinds of triads, it is completely wrong by overestimating some and underestimating others. It makes it clear that some important terms for the development of the network are still left out and not being considered.

3.1.5

```
source("printSiena.R")
model_saom
```

```
## Estimates, standard errors and convergence t-ratios
```

```
##
##
##          Estimate      Standard      Convergence
##          Error        t-ratio
##
## Rate parameters:
## 0.1      Rate parameter period 1    12.3619 ( 1.3812 )
## 0.2      Rate parameter period 2     9.7971 ( 0.9900 )
##
## Other parameters:
## 1. eval outdegree (density)        -2.5681 ( 0.1455 )  -0.0483
## 2. eval reciprocity                2.2687 ( 0.0867 )  -0.0466
## 3. eval 3-cycles                   1.0295 ( 0.0537 )  -0.0450
## 4. eval indegree - popularity (sqrt) 0.0056 ( 0.0689 )  -0.0554
## 5. eval logdistance                -0.1852 ( 0.0447 )  -0.0005
## 6. eval alcohol alter              0.0017 ( 0.0314 )   0.0367
## 7. eval alcohol ego                -0.0561 ( 0.0385 )  -0.0479
## 8. eval alcohol similarity          0.4939 ( 0.1650 )  -0.0160
##
## Overall maximum convergence ratio: 0.1158
##
##
## Degrees constrained to maximum values:
## friendship : 6
##
##
## Total of 4510 iteration steps.
```

```
printSiena(model_saom)
```

```
##      dependent          effect  theta  s.e. p.value sig.
## 1      rate constant friendship rate (period 1) 12.362 1.381
## 2      rate constant friendship rate (period 2) 9.797 0.990
## 3 friendship          outdegree (density) -2.568 0.145      0 ***
## 4 friendship          reciprocity 2.269 0.087      0 ***
## 5 friendship          3-cycles 1.029 0.054      0 ***
## 6 friendship      indegree - popularity (sqrt) 0.006 0.069 0.935
## 7 friendship          logdistance -0.185 0.045      0 ***
## 8 friendship          alcohol alter 0.002 0.031 0.956
## 9 friendship          alcohol ego -0.056 0.039 0.145
## 10 friendship      alcohol similarity 0.494 0.165 0.003 **
##      t.conv
## 1
## 2
## 3 -0.048
## 4 -0.047
## 5 -0.045
## 6 -0.055
## 7 0
## 8 0.037
## 9 -0.048
## 10 -0.016
```

Hypothesis (i) Students tend to be friends with popular pupils: This hypothesis is not supported by the SAOM model. The extremely high p-value 0.935 shows no significance, along with the near zero coefficient

+0.006, showing that even if it had significance, the effect would be barely noticeable. Since SAOM estimate how the network changes, it just says the popularity plays no role in the formation of new ties. A student will nominate a person regardless of how popular that student already is.

Hypothesis (ii) Students tend to be friends with pupils with similar alcohol consumption to theirs: With a p-value of 0.003, this hypothesis is significant and with a coefficient value of +0.494 is supported. The likelihood of a tie forming between two students at the same alcohol consumption level is $e^{0.494} \approx 1.64$ times higher than if they had the biggest possible difference. This means that alcohol homophily is an important factor in the development of the structural network and that friendships will center partly around the alcohol consumption level of students. This means that alcohol consumption contributes to cluster formation around alcohol homophily. It is to be noted that both alcohol alter and alcohol ego have high p-values and low total coefficients, meaning that alcohol consumption has no noticeable effect on the amount of nominations a student will send or receive.

Hypothesis (iii) Students tend to be friends with students who live in the same neighborhood (living nearby): The values of logdistance support this hypothesis. With a near zero p-value and a coefficient of -0.185 , the simulated model predicts that the further away two students live from each other, the less likely it becomes that they nominate one another. Thus they are more likely to nominate someone who lives close to them. Similar to alcohol homophily, this means that the geographical position of students homes will lead to some clustering over enough time. So if the network has its nodes centered on a map, it should show clear clustering around certain areas like different neighborhoods.

3.2

3.2.1

```
# Alcohol as dependent behaviour variable
alcoholbeh <- as_dependent_rsiena(
  as.matrix(alcohol_data[, 1:3]),
  type = "behavior"
)

# Rebuild data object with alcohol as dependent variable
mydata_coev <- make_data_rsiena(friendship, alcoholbeh, gender, logdistance)

# Check preconditions

write_report(mydata_coev, outputName = "glasgowCoevReport")

# Moran's I for autocorrelation in alcohol behaviour
moran1 <- nacf(f1, alcohol_data[, 1], lag.max = 1, type = "moran",
  neighborhood.type = "out", mode = "digraph")
moran2 <- nacf(f2, alcohol_data[, 2], lag.max = 1, type = "moran",
  neighborhood.type = "out", mode = "digraph")
moran3 <- nacf(f3, alcohol_data[, 3], lag.max = 1, type = "moran",
  neighborhood.type = "out", mode = "digraph")
autocorr <- rbind(moran1, moran2, moran3)
autocorr[, 2]

##      moran1      moran2      moran3
## 0.2450651 0.3394460 0.3316509
```

```

## Model specification
myeff_coev <- make_specification(mydata_coev)

# --- Selection part: identical to 3.1.2 ---
myeff_coev <- set_effect(myeff_coev, list( recip),          depvar = "friendship")

##   effectNumber effectName  shortName include fix   test  initialValue parm
## 1 17            reciprocity recip      TRUE  FALSE FALSE           0  0

myeff_coev <- set_effect(myeff_coev, list(transTrip),      depvar = "friendship")

##   effectNumber effectName          shortName include fix   test  initialValue
## 1 23            transitive triplets transTrip TRUE  FALSE FALSE           0
##   parm
## 1 0

myeff_coev <- set_effect(myeff_coev, list(cycle3),          depvar = "friendship")

##   effectNumber effectName shortName include fix   test  initialValue parm
## 1 43            3-cycles  cycle3      TRUE  FALSE FALSE           0  0

myeff_coev <- set_effect(myeff_coev, list(egoX, altX, sameX), depvar = "friendship",
                          covar1 = "gender")

##   effectNumber effectName  shortName include fix   test  initialValue parm
## 1 297            gender alter altX      TRUE  FALSE FALSE           0  0
## 2 312            gender ego  egoX      TRUE  FALSE FALSE           0  0
## 3 368            same gender sameX      TRUE  FALSE FALSE           0  0

# Hypothesis i: popularity
myeff_coev <- set_effect(myeff_coev, list(inPopSqrt),      depvar = "friendship")

##   effectNumber effectName          shortName include fix   test
## 1 86            indegree - popularity (sqrt) inPopSqrt TRUE  FALSE FALSE
##   initialValue parm
## 1 0 0

# Hypothesis ii: alcohol homophily (now using the behaviour dependent variable)
myeff_coev <- set_effect(myeff_coev, list(egoX, altX, simX), depvar = "friendship",
                          covar1 = "alcoholbeh")

##   effectNumber effectName          shortName include fix   test  initialValue
## 1 481            alcoholbeh alter altX      TRUE  FALSE FALSE           0
## 2 496            alcoholbeh ego  egoX      TRUE  FALSE FALSE           0
## 3 542            alcoholbeh similarity simX      TRUE  FALSE FALSE           0
##   parm
## 1 0
## 2 0
## 3 0

```



```

# Hypothesis iii: geographic proximity
myeff_coev <- set_effect(myeff_coev, list(X), depvar = "friendship",
                        covar1 = "logdistance")

##   effectNumber effectName  shortName include fix  test  initialValue parm
## 1 264          logdistance X          TRUE  FALSE FALSE          0    0

# --- Influence part: effects explaining alcohol behaviour evolution ---

# Basic controls for behaviour (linear and quadratic shape)
# These are included by default in make_specification but listed here for clarity
# They capture the baseline tendency to increase/decrease alcohol consumption

# Hypothesis iv: Popular students tend to increase or maintain alcohol consumption
# indeg: students who receive more friendship nominations (more popular)
# tend to increase their alcohol consumption
myeff_coev <- set_effect(myeff_coev, list(indeg), depvar = "alcoholbeh",
                        covar1 = "friendship")

##   effectNumber effectName          shortName include fix  test  initialValue
## 1 792          alcoholbeh indegree indeg      TRUE  FALSE FALSE          0
##   parm
## 1 0

# Hypothesis v: Students adjust alcohol consumption to that of their friends
# avSim: average similarity effect - ego moves their behaviour toward
# the average alcohol level of their alters (the core social influence effect)
myeff_coev <- set_effect(myeff_coev, list(avSim), depvar = "alcoholbeh",
                        covar1 = "friendship")

##   effectNumber effectName          shortName include fix  test
## 1 755          alcoholbeh average similarity avSim      TRUE  FALSE FALSE
##   initialValue parm
## 1          0    0

myeff_coev

##   name          effectName          include fix  test
## 1  friendship constant friendship rate (period 1) TRUE  FALSE FALSE
## 2  friendship constant friendship rate (period 2) TRUE  FALSE FALSE
## 3  friendship outdegree (density)          TRUE  FALSE FALSE
## 4  friendship reciprocity                  TRUE  FALSE FALSE
## 5  friendship transitive triplets          TRUE  FALSE FALSE
## 6  friendship 3-cycles                    TRUE  FALSE FALSE
## 7  friendship indegree - popularity (sqrt) TRUE  FALSE FALSE
## 8  friendship logdistance                  TRUE  FALSE FALSE
## 9  friendship gender alter                 TRUE  FALSE FALSE
## 10 friendship gender ego                   TRUE  FALSE FALSE
## 11 friendship same gender                  TRUE  FALSE FALSE
## 12 friendship alcoholbeh alter             TRUE  FALSE FALSE
## 13 friendship alcoholbeh ego               TRUE  FALSE FALSE

```

## 14	friendship alcoholbeh similarity	TRUE	FALSE	FALSE
## 15	alcoholbeh rate alcoholbeh (period 1)	TRUE	FALSE	FALSE
## 16	alcoholbeh rate alcoholbeh (period 2)	TRUE	FALSE	FALSE
## 17	alcoholbeh alcoholbeh linear shape	TRUE	FALSE	FALSE
## 18	alcoholbeh alcoholbeh quadratic shape	TRUE	FALSE	FALSE
## 19	alcoholbeh alcoholbeh average similarity	TRUE	FALSE	FALSE
## 20	alcoholbeh alcoholbeh indegree	TRUE	FALSE	FALSE
##	initialValue parm			
## 1	7.45424 0			
## 2	6.82927 0			
## 3	-1.61299 0			
## 4	0.00000 0			
## 5	0.00000 0			
## 6	0.00000 0			
## 7	0.00000 0			
## 8	0.00000 0			
## 9	0.00000 0			
## 10	0.00000 0			
## 11	0.00000 0			
## 12	0.00000 0			
## 13	0.00000 0			
## 14	0.00000 0			
## 15	0.91000 0			
## 16	1.09314 0			
## 17	0.34704 0			
## 18	0.00000 0			
## 19	0.00000 0			
## 20	0.00000 0			

3.2.2

```

# first step model estimation
SAOMAlgorithm_coev <- sienaAlgorithmCreate(seed = 161, n3 = 3000, nsub = 4,
                                           MaxDegree = c(friendship = 6))

model_coev <- siena07(SAOMAlgorithm_coev,
                      data      = mydata_coev,
                      effects   = myeff_coev,
                      returnDeps = TRUE,
                      batch     = FALSE)

# second step model estimation
SAOMAlgorithm_converge <- sienaAlgorithmCreate(
  seed = 161,
  n3 = 5000,
  nsub = 5,
  MaxDegree = c(friendship = 6)
)

model_coev_final <- siena07(
  SAOMAlgorithm_converge,
  data = mydata_coev,

```

```

effects = myeff_coev,
prevAns = model_coev, # Using previous failed run as the starting point
returnDeps = TRUE,
batch = FALSE
)

#model_coev_final

# save model
saveRDS(model_coev_final, "model_coef_final.rds")

model_coev_final <- readRDS("model_coef_final.rds")
# Convergence check
t_ratios_coev <- apply(model_coev_final$sf, 2, mean) / apply(model_coev_final$sf, 2, sd)
t_ratios_coev

## [1] -0.0135874937 -0.0086024967 0.0300746609 0.0316140308 0.0165487528
## [6] 0.0093000455 0.0339679526 0.0346385251 -0.0180897629 -0.0082636378
## [11] 0.0314324650 0.0079810074 -0.0037956706 0.0005432766 0.0269788564
## [16] 0.0168935341 0.0068595218 -0.0099553750 -0.0147690147 0.0081376447

overall_coev <- sqrt(t(apply(model_coev_final$sf, 2, mean)) %*%
                     solve(cov(model_coev_final$sf)) %*%
                     apply(model_coev_final$sf, 2, mean))
overall_coev

## [1,]
## [1,] 0.09271135

```

As can be seen from these results, all absolute values of the t-ratios are below 0.1 and additionally the overall value also lies below 0.25. This fulfills the convergence criterion, indicating that all of our chosen parameters were sufficient to achieve convergence. The SAOM was estimated with 5 subphases and 5000 iterations in phase 3, which was enough to achieve this convergence and did not need to have another re-estimation or other model adjustments. These parameters consist of the three needed for answering our hypothesis: one parameter to control the popularity of a student by checking for the indegree-popularity value, another to check for alcohol consumption similarity and the last one to see if geographic proximity has an effect on the model evolution. For the influence part of the model we included average similarity and indegree effects on the alcohol behavior dependent variable. Furthermore, we included basic endogenous and exogenous variables that play a significant part in the structure of most networks: variables to check for reciprocity, transitive triplets, and 3-cyclic patterns. We also added a parameter to control for same-gender homophily, as we have seen that gender plays a major role in student network structures.

3.2.3

```

## Goodness of fit
cl <- makeCluster(4)

gof_coev_id <- test_gof(model_coev_final, verbose = FALSE,
                        varName = "friendship", IndegreeDistribution, cluster = cl)
gof_coev_od <- test_gof(model_coev_final, verbose = FALSE,

```

```

        varName = "friendship", OutdegreeDistribution, cluster = c1)
gof_coev_tc <- test_gof(model_coev_final, verbose = FALSE,
        varName = "friendship", TriadCensus, cluster = c1)
gof_coev_gd <- test_gof(model_coev_final, verbose = FALSE,
        varName = "friendship", GeodesicDistribution)
gof_coev_beh <- test_gof(model_coev_final, verbose = FALSE,
        varName = "alcoholbeh", BehaviorDistribution, cluster = c1)

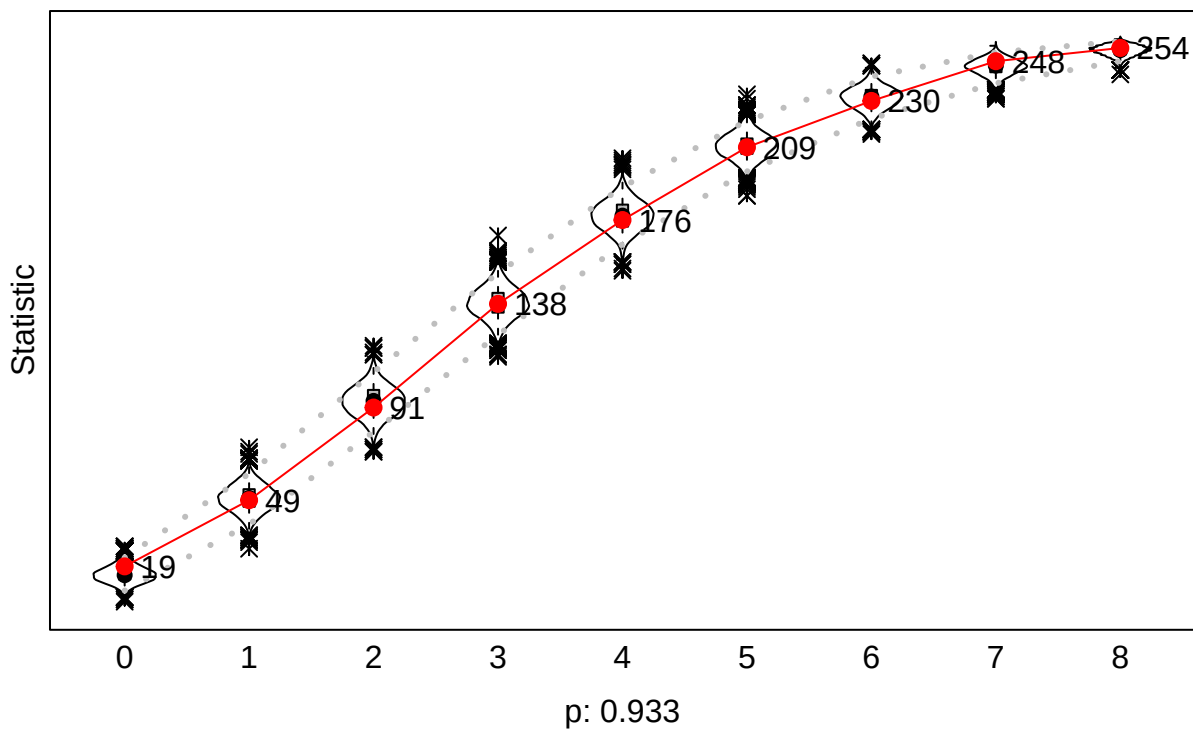
stopCluster(c1)
saveRDS(gof_coev_id, "gof_coev_id.rds")
saveRDS(gof_coev_od, "gof_coev_od.rds")
saveRDS(gof_coev_gd, "gof_coev_gd.rds")
saveRDS(gof_coev_tc, "gof_coev_tc.rds")
saveRDS(gof_coev_beh, "gof_coev_beh.rds")

gof_coev_id <- readRDS("gof_coev_id.rds")
gof_coev_od <- readRDS("gof_coev_od.rds")
gof_coev_gd <- readRDS("gof_coev_gd.rds")
gof_coev_tc <- readRDS("gof_coev_tc.rds")
gof_coev_beh <- readRDS("gof_coev_beh.rds")

plot(gof_coev_id)

```

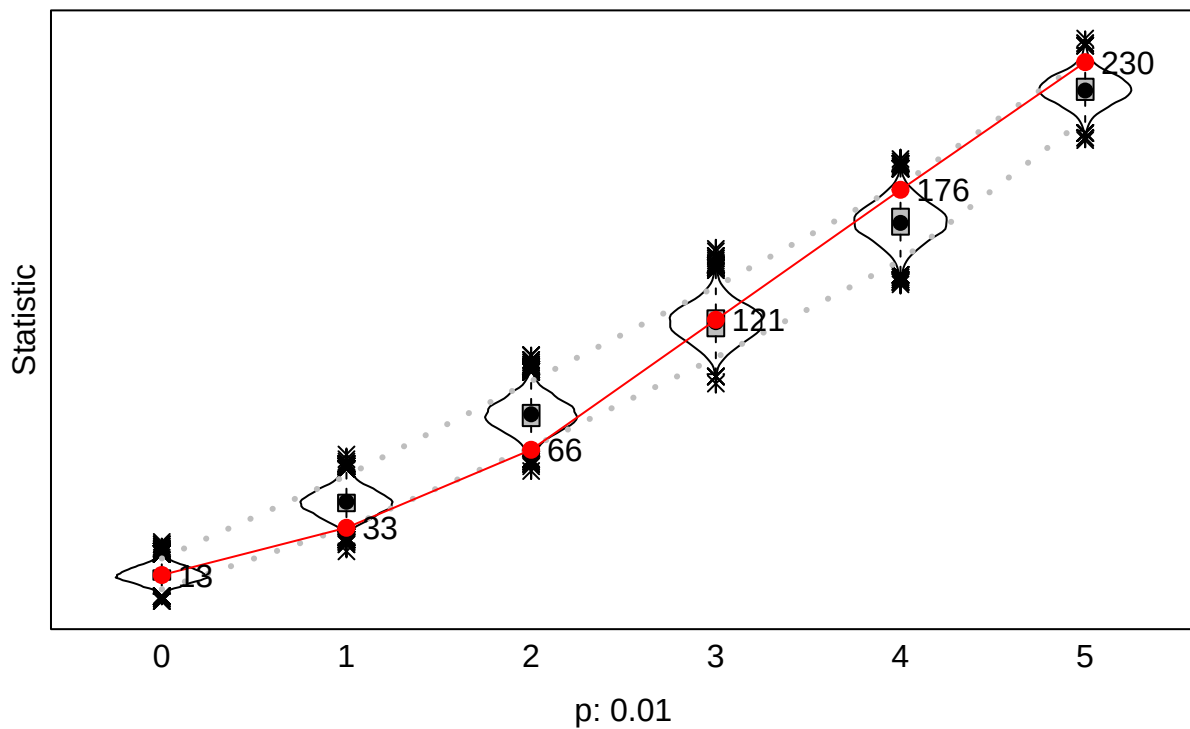
Goodness of Fit of IndegreeDistribution



```
plot(gof_coev_od)
```

```
## Note: some statistics are not plotted because their variance is 0.  
## This holds for the statistics: 6 7 8.
```

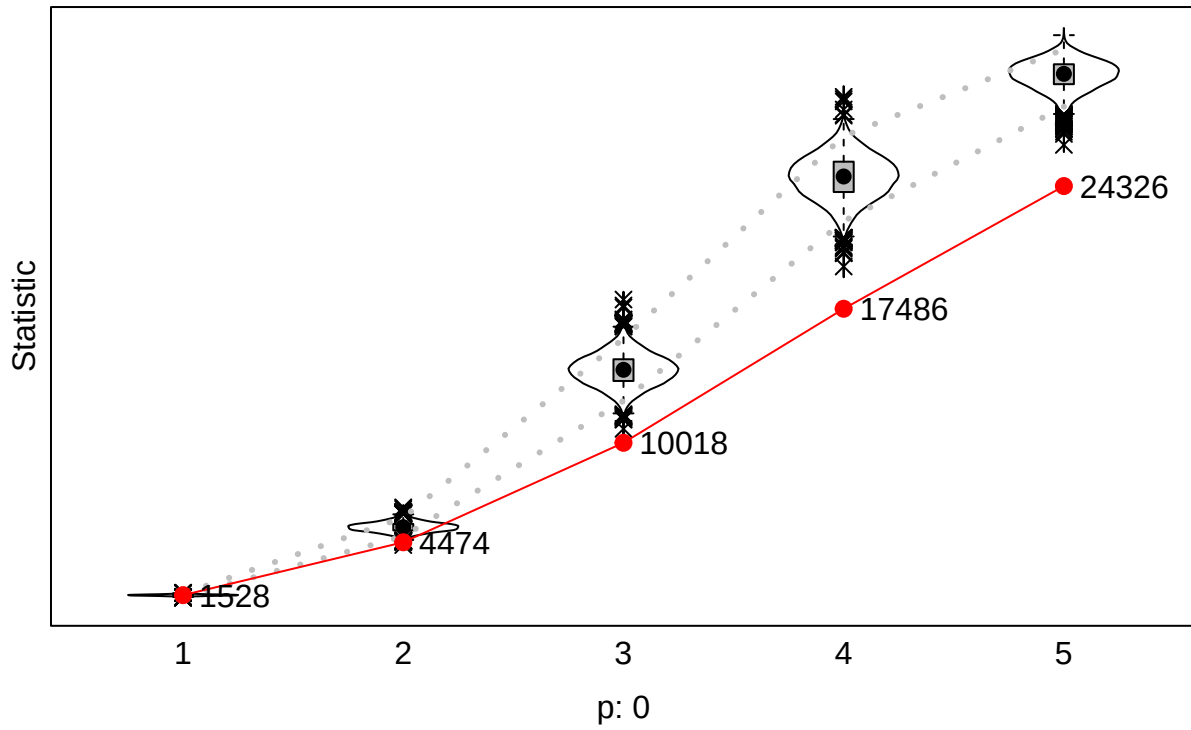
Goodness of Fit of OutdegreeDistribution



```
plot(gof_coev_gd)
```

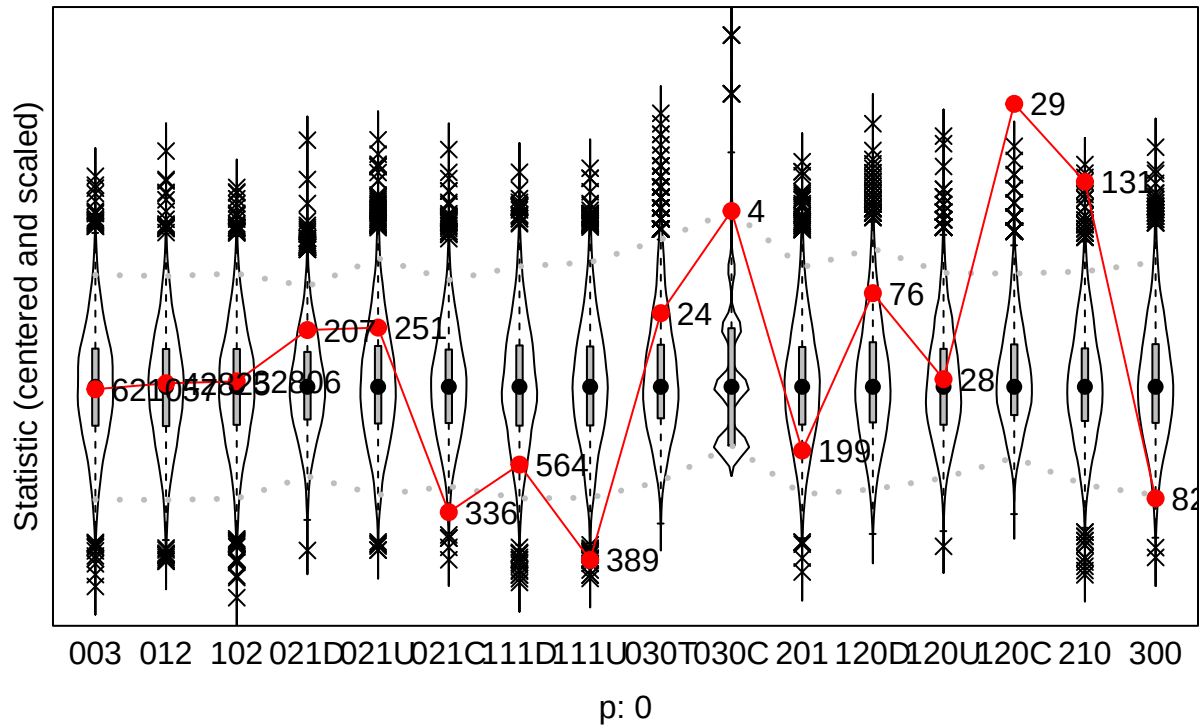
```
## Note: some statistics are not plotted because their variance is 0.  
## This holds for the statistic: Inf.
```

Goodness of Fit of GeodesicDistribution



```
plot(gof_coev_tc, center = TRUE, scale = TRUE)
```

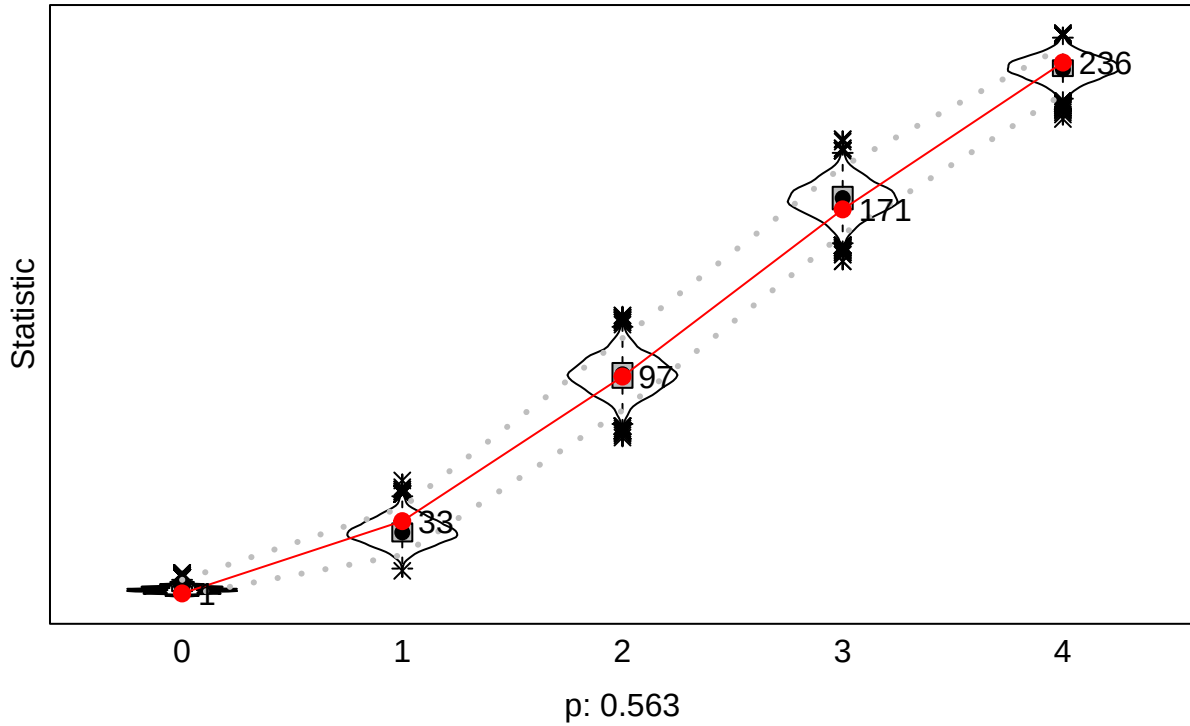
Goodness of Fit of TriadCensus



```
plot(gof_coev_beh)
```

```
## Note: some statistics are not plotted because their variance is 0.  
## This holds for the statistic: 5.
```

Goodness of Fit of BehaviorDistribution



#model_coev_final

The plot for the goodness of fit for the indegree distribution shows that our SAOM model captures this property very well. The plot follows the real data and as such produces a p-value of 0.933, which supports the fact that the model captures this property well. This is because high p-values close to 1 support the property being captured. Additionally, the behavior distribution for alcohol consumption is captured effectively with a p-value of 0.563, indicating the influence part of the model is performing reasonably well. The Outdegree distribution is at a low p-value of 0.01, which means it does not capture this property well. This is despite implementing the maximum outdegree constraint of 6 per student as the data collection method enforces, meaning that while the constraint is very important to reflect the observed network's limits, there are still some other variables important for the development of the network that were not accounted for so far. The other two plotted parameters are both not captured well. To start of with, they both have a p-value of 0, which means that neither of them accurately capture the observed data. The geodesic distribution itself has values that are higher than they are supposed to be over all distances, granting it a p-value of 0. This means that the simulated network is more connected than the real one, and that the real one has more of a community type structured one, meaning there are more separate clusters that have only few edges going out / coming in. The Triad consensus also has a p-value of 0, and as seen in the graph, while it correctly predicts some kinds of triads, it is completely wrong by overestimating some and underestimating others. It makes it clear that some important terms for the development of the network are still left out and not being considered.

3.2.4

Hypothesis iv: Popular students tend to increase or maintain their level of alcohol consumption: This hypothesis is not supported by the data. The estimate for the effect of indegree on alcoholbeh is 0.0703,

which is positive but statistically insignificant given the standard error of 0.0645. Because the estimate is not at least twice the standard error, we can't confidently conclude that popularity in the friendship network drives alcohol consumption. While the direction suggest popular students might consume more, the effect is not statistically distinguishable from zero. Hypothesis v: Students adjust their alcohol level to match their friends: This hypothesis is strongly supported, as the *alcoholbeh* average similarity effect has a high positive coefficient of 6.9119 and is statistically significant, as it has a standard error of 1.8432. This indicates that students change their drinking habits to minimize the distance between themselves and the average consumption level of their friends. The high behavior distribution with a p-value of 0.502 confirms that the model accurately captures these behavioral shifts.

Revisiting hypotheses i-iii: Hypothesis i: Students tend to be friends with popular pupils: In the co-evolution model, the indegree-popularity effect is now negative with a value of -0.2400 and significant as its standard error is 0.0761. This differs significantly from the finding in (1), where the effect was nearly zero and insignificant. This indicates that when we are accounting for alcohol co-evolution, students actually show a slight preference against forming ties with the most popular peers. Hypothesis ii: Students tend to be friends with pupils with similar alcohol consumption to theirs: The support for this hypothesis remains strong, but the magnitude has increased significantly. The *alcoholbeh* similarity coefficient rose from 0.494 in (1) to 1.1924 now. This suggests that once we account for the fact that friends influence each others drinking habits, the tendency to choose friends based on pre-existing alcohol similarity appears to be even stronger than previously thought. Hypothesis iii: Students tend to be friends with students who live in the same neighborhood (living nearby): For this hypothesis the findings remain consistent and significant. The *logdistance* coefficient is -0.1950 with a standard error of 0.0461, which closely matches the -0.185 we found in (1). The negative value confirms that as the distance between homes increases, the likelihood of friendship decreases, which supports our hypothesis that students from the same neighborhoods tend to be friends. However the geodesic distribution still has a p-value of 0, which tells us that while the distance of students is important, it is not sufficient to fully explain the network's overall community structure.

3.2.5

We have strong evidence for both selection and influence processes operating simultaneously within the co-evolution of the friendship network and alcohol consumption. Selection processes can be seen by the *alcoholbeh* similarity in the Network Dynamics part of the model, which yields a positive estimate of 1.1924 with a standard error of 0.3403. Since this coefficient is more than three times its standard error, it is statistically significant, which indicates that students actively select friends who have similar drinking habits to their own. This selection based on behavioral homophily is a key driver of the friendship network's structural evolution. Next to the selection processes the model also provides strong evidence for influence processes in the Behavior Dynamics part of the model. Here it shows a large estimate of 6.9119, that is statistically significant with a standard error of 1.8432, which confirms that students do not just choose similar friends, but also adapt their own alcohol consumption over time to align with the average behavior of their existing friend group (as also discussed in Task 3.2.4).

3.3

There are more nodes with a low geodesic distance in the simulated model than in the real observed one, meaning that the simulated one does not create as strongly isolated clusters as the observed model has. A simulated model could achieve better scores here, if it encourages isolated clusters more. This means we would require that the current transitivity effect becomes stronger. At the same time, we need the students to be more likely to cut a tie when they do not share many common friends and are not connected well to the node otherwise. This should lead to stronger community forming. To further get all the effects on transitivity, we should add the *transRecTrip* and the *qwesp* terms.

The indegree distribution already closely follows the real world model, so it does not require any changes to it, since it currently captures the model very well.

The outdegree distribution could be more accurate. It has too many students with a low outdegree, and too few students with a high outdegree. This too can be due to a too weak community clustering, although that should have a rather small effect on it, the model might just have an overall too high penalty for outgoing ties which would need to be reduced. We might need to include an additional parameter like *outAct* to capture more details on the network structure.

3.4

An additional hypothesis to further look into the co-evolution of social structures and alcohol consumption could be that the relationship between alcohol consumption and popularity is moderated by gender, suggesting that a high alcohol consumption increases the popularity of males but decreases it for females. To operationalize the corresponding effects in the evaluation function, one would collect gender as a binary attribute and specify a behavior-related popularity effect interacted with a gender-ego effect. This tests if the “social reward” of drinking depends on the gender and would not treat all students as a homogeneous group, as there are now genders associated with each student. A second additional hypothesis to be tested would be that the influence of peers on alcohol consumption is influenced by the frequency of their physical interactions outside of school. More specifically this means that students are more likely to assimilate to the alcohol levels of friends with whom they have a lot of physical interactions. This could be operationalized by collecting the contact frequency of friends as a dyadic covariate and implementing a weighted average similarity effect in the behavior dynamics part. Rather than the broad alcoholbeh average similarity coefficient, this would weight the influence of each friend by the reported frequency of their interactions.